

บทที่ 2

ปัญญาประดิษฐ์และการแก้ปัญหา

หนึ่งในเป้าหมายสำคัญของปัญญาประดิษฐ์ (Artificial Intelligence: AI) คือการพัฒนาระบบที่สามารถตัดสินใจและแก้ปัญหาได้อย่างชาญฉลาดคล้ายมนุษย์ การตัดสินใจดังกล่าวจำเป็นต้องอาศัยการรับรู้สภาพแวดล้อม การประเมินสถานการณ์ และการเลือกการกระทำที่เหมาะสมที่สุดภายใต้ข้อมูลที่มีอยู่ในขณะนั้น แนวคิดพื้นฐานนี้นำไปสู่การพัฒนาตัวแทนปัญญาประดิษฐ์ (Intelligent Agent) ซึ่งเป็นองค์ประกอบสำคัญของระบบ AI สมัยใหม่

ตัวแทนปัญญาประดิษฐ์สามารถพบได้ในชีวิตประจำวันหลายรูปแบบ เช่น ระบบนำทางในรถยนต์ ระบบแนะนำสินค้า ระบบวินิจฉัยโรค หรือหุ่นยนต์อัตโนมัติ โดยตัวแทนเหล่านี้มีหน้าที่รับข้อมูลจากสภาพแวดล้อม ประมวลผลข้อมูล และเลือกการกระทำที่คาดว่าจะให้ผลลัพธ์ที่ดีที่สุดตามเป้าหมายที่กำหนดไว้ แนวคิดดังกล่าวเรียกว่า ตัวแทนที่มีเหตุผล (Rational Agent)

นอกจากการออกแบบตัวแทนแล้ว การแก้ปัญหา (Problem Solving) ยังเป็นหัวข้อสำคัญของปัญญาประดิษฐ์ โดยปัญหาต่าง ๆ จำเป็นต้องถูกแทนให้อยู่ในรูปแบบที่คอมพิวเตอร์สามารถวิเคราะห์และประมวลผลได้ จากนั้นจึงนำขั้นตอนวิธีการค้นหา (Search Algorithms) มาใช้ในการค้นหาคำตอบหรือเส้นทางที่ดีที่สุดภายในปริภูมิสถานะ (State Space)

การค้นหาในปัญญาประดิษฐ์สามารถแบ่งออกได้เป็นสองกลุ่มหลัก ได้แก่ การค้นหาแบบไม่ใช้ข้อมูลนำทาง (Uninformed Search) ซึ่งอาศัยเพียงข้อมูลของปัญหา และการค้นหาแบบใช้ข้อมูลนำทาง (Informed Search) ซึ่งใช้ข้อมูลเพิ่มเติมหรือฟังก์ชันฮิวริสติก (Heuristic Function) เพื่อช่วยเพิ่มประสิทธิภาพในการค้นหา ทำให้สามารถลดจำนวนสถานะที่ต้องพิจารณาและค้นหาคำตอบได้รวดเร็วยิ่งขึ้น

บทนี้จะอธิบายแนวคิดของตัวแทนที่มีเหตุผล หลักการแก้ปัญหา การแทนปัญหาให้อยู่ในรูปแบบมาตรฐาน ปัญหาตัวอย่างในงานปัญญาประดิษฐ์ ตลอดจนหลักการของขั้นตอนวิธีการค้นหาแบบต่าง ๆ ซึ่งถือเป็นรากฐานสำคัญของปัญญาประดิษฐ์แบบดั้งเดิม (Classical Artificial Intelligence) และเป็นพื้นฐานสำหรับการศึกษาเทคนิคขั้นสูงในบทต่อไป

2.1 ตัวแทนและตัวแทนที่มีเหตุผล

ปัญญาประดิษฐ์ (Artificial Intelligence: AI) มีเป้าหมายสำคัญคือการพัฒนาระบบที่สามารถรับรู้สภาพแวดล้อม วิเคราะห์ข้อมูล และเลือกการกระทำที่เหมาะสมเพื่อบรรลุเป้าหมายที่กำหนดไว้ ระบบดังกล่าวถูกเรียกว่า ตัวแทน (Agent) ซึ่งเป็นแนวคิดพื้นฐานที่สำคัญของปัญญาประดิษฐ์สมัยใหม่

ในชีวิตประจำวัน มนุษย์เองก็สามารถถูกมองว่าเป็นตัวแทนชนิดหนึ่ง กล่าวคือ มนุษย์รับข้อมูลจากสิ่งแวดล้อมผ่านอวัยวะรับสัมผัส เช่น ตา หู จมูก ลิ้น และผิวหนัง จากนั้นสมองจะทำการประมวลผลและ

ตัดสินใจแสดงพฤติกรรมหรือการกระทำผ่านอวัยวะต่าง ๆ เช่น มือ เท้า หรือการพูด เช่นเดียวกับระบบปัญญาประดิษฐ์ที่มีการรับข้อมูลและตอบสนองต่อสิ่งแวดล้อมเพื่อให้บรรลุวัตถุประสงค์ที่กำหนดไว้

2.1.1 ตัวแทน (Agent)

Russell และ Norvig (2021) ให้นิยามของตัวแทน (Agent) ว่า “Anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.” กล่าวคือ ตัวแทน หมายถึง สิ่งใดก็ตามที่สามารถรับรู้สภาพแวดล้อมผ่านตัวรับข้อมูล (Sensors) และตอบสนองต่อสภาพแวดล้อมผ่านตัวกระทำ (Actuators)

ดังนั้น กระบวนการทำงานพื้นฐานของตัวแทนสามารถแบ่งออกเป็น 3 ส่วน ได้แก่

1. สภาพแวดล้อม (Environment)

คือสิ่งต่าง ๆ ที่อยู่รอบล้อมตัวแทน ซึ่งอาจเป็นโลกทางกายภาพหรือโลกเสมือน

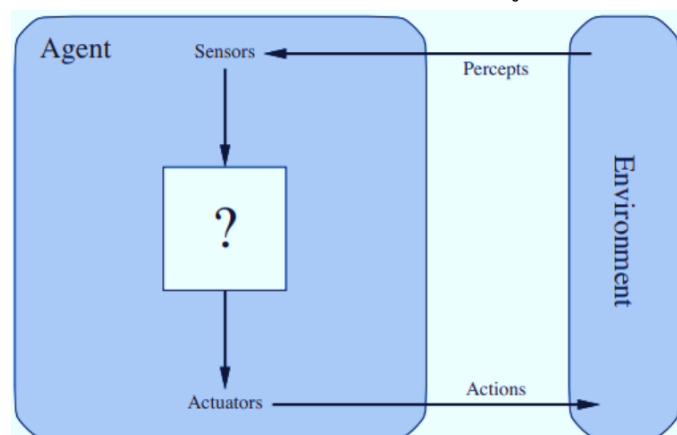
2. ตัวรับข้อมูล (Sensors)

ทำหน้าที่รับข้อมูลจากสภาพแวดล้อมเข้าสู่ระบบ

3. ตัวกระทำ (Actuators)

ทำหน้าที่แสดงผลหรือกระทำต่อสภาพแวดล้อม

ความสัมพันธ์ระหว่างองค์ประกอบทั้งสามสามารถแสดงได้ดังรูปที่ 2.1



รูปที่ 2.1 โครงสร้างพื้นฐานของตัวแทน (Agent)

จากรูปที่ 2.1 ตัวแทนจะรับข้อมูลจากสภาพแวดล้อมผ่านตัวรับข้อมูล จากนั้นจึงประมวลผลข้อมูล และเลือกการกระทำที่เหมาะสม ก่อนส่งผลลัพธ์กลับไปยังสภาพแวดล้อมผ่านตัวกระทำ กระบวนการนี้จะเกิดขึ้นอย่างต่อเนื่องเป็นวงจร

2.1.2 ตัวรับข้อมูล

ตัวรับข้อมูล (Sensors) คืออุปกรณ์หรือกลไกที่ใช้ในการรับข้อมูลจากสภาพแวดล้อมเข้าสู่ตัวแทน โดยตัวรับข้อมูลจะแตกต่างกันไปตามลักษณะของระบบ

ตารางที่ 2.1 ตัวอย่างของตัวรับข้อมูลของตัวแทน

ตัวแทน	ตัวรับข้อมูล (Sensors)
มนุษย์	ตา หู จมูก ลิ้น ผิวหนัง
รถยนต์ขับเคลื่อนอัตโนมัติ	กล้อง เรดาร์ LiDAR GPS เซนเซอร์วัดระยะ
ระบบวินิจฉัยโรค	ข้อมูลผู้ป่วย ผลตรวจเลือด ภาพเอกซเรย์
หุ่นยนต์ดูดฝุ่น	เซนเซอร์ตรวจจับสิ่งกีดขวาง กล้อง อินฟราเรด
โปรแกรมเล่นหมากรุก	ตำแหน่งหมากบนกระดาน การเดินของคู่แข่ง

2.1.3 ตัวกระทำ (Actuators)

ตัวกระทำ (Actuators) คืออุปกรณ์หรือกลไกที่ใช้ในการแสดงพฤติกรรมหรือกระทำต่อสภาพแวดล้อม ภายหลังจากที่ตัวแทนได้ทำการประมวลผลแล้ว ตัวอย่างของตัวกระทำแสดงดังตารางที่ 2.2

ตารางที่ 2.2 ตัวอย่างของตัวกระทำของตัวแทน

ตัวแทน	ตัวกระทำ (Actuators)
มนุษย์	มือ เท้า การพูด
รถยนต์ขับเคลื่อนอัตโนมัติ	พวงมาลัย เบรก คันเร่ง ไฟเลี้ยว
ระบบวินิจฉัยโรค	รายงานผล คำแนะนำการรักษา
หุ่นยนต์ดูดฝุ่น	ล้อ มอเตอร์ เครื่องดูดฝุ่น
โปรแกรมเล่นหมากรุก	การเลือกตำแหน่งเดินหมาก

2.1.4 ตัวแทนที่มีเหตุผล (Rational Agent)

แนวคิดที่สำคัญที่สุดในปัญญาประดิษฐ์คือ ตัวแทนที่มีเหตุผล (Rational Agent)

ตัวแทนที่มีเหตุผล หมายถึง ตัวแทนที่เลือกการกระทำที่คาดว่าจะให้ผลลัพธ์ที่ดีที่สุดจากข้อมูลที่มีอยู่ ณ เวลานั้น กล่าวอีกนัยหนึ่ง ตัวแทนที่มีเหตุผลไม่ได้หมายถึงตัวแทนที่สามารถทำนายอนาคตได้อย่างสมบูรณ์ แต่เป็นตัวแทนที่สามารถตัดสินใจได้ดีที่สุดจากข้อมูลที่รับรู้ได้ในปัจจุบัน

หลักการดังกล่าวสามารถเขียนได้ดังนี้

เลือกการกระทำ (Action) ที่ทำให้ค่าคาดหวังของประสิทธิภาพ (Expected Performance) สูงที่สุด

กล่าวคือ Rational Agent = Maximize Expected Performance

ตัวอย่างเช่น การข้ามถนนของมนุษย์ ก่อนข้ามถนน มนุษย์จะมองซ้ายและมองขวาเพื่อประเมินว่ามีรถกำลังวิ่งมาหรือไม่ หากไม่พบรถอยู่ในระยะอันตรายจึงตัดสินใจข้ามถนน แม้ว่าจะเกิดอุบัติเหตุขึ้นในภายหลัง การตัดสินใจดังกล่าวก็ยังถือว่าเป็นการตัดสินใจอย่างมีเหตุผล เพราะใช้ข้อมูลที่มีอยู่ในขณะนั้นอย่างเหมาะสมที่สุด

ดังนั้น การเป็นตัวแทนที่มีเหตุผลไม่ได้หมายความว่าต้องให้ผลลัพธ์ถูกต้องเสมอไป แต่หมายถึงการตัดสินใจโดยอาศัยข้อมูลที่มีอยู่ ณ เวลานั้นอย่างดีที่สุด

2.1.5 ความแตกต่างระหว่าง Rationality และ Omniscience

ผู้เริ่มศึกษาปัญญาประดิษฐ์มักเข้าใจผิดว่าตัวแทนที่มีเหตุผลจะต้องสามารถตัดสินใจได้ถูกต้องทุกครั้ง อย่างไรก็ตาม ในทางปฏิบัติไม่มีระบบใดสามารถรู้ข้อมูลทุกอย่างล่วงหน้าได้

แนวคิดที่สามารถรู้ทุกสิ่งทุกอย่างและทราบผลลัพธ์ในอนาคตเรียกว่า Omniscience หรือความรู้สมบูรณ์

ตารางที่ 2.3 เปรียบเทียบระหว่าง Rationality และ Omniscience

หัวข้อ	Rationality	Omniscience
พื้นฐานการตัดสินใจ	ใช้ข้อมูลที่ได้รับรู้ได้	ใช้ข้อมูลทั้งหมด
ความรู้เกี่ยวกับอนาคต	ไม่สามารถรู้ล่วงหน้า	รู้ล่วงหน้าได้ทั้งหมด
เป้าหมาย	ให้ผลลัพธ์ที่ดีที่สุดที่คาดหวัง	ให้ผลลัพธ์ที่ดีที่สุดจริงเสมอ
ความเป็นไปได้ในโลกจริง	สามารถเกิดขึ้นได้	ไม่สามารถเกิดขึ้นได้จริง

ดังนั้น เป้าหมายของปัญญาประดิษฐ์ไม่ได้มุ่งสร้างระบบที่มีความรู้สมบูรณ์ (Omniscient System) แต่เป็นการสร้างตัวแทนที่สามารถตัดสินใจได้อย่างมีเหตุผล (Rational Agent) ภายใต้ข้อจำกัดของข้อมูล เวลา และทรัพยากรที่มีอยู่

2.1.6 วงจรการทำงานของตัวแทนที่มีเหตุผล

กระบวนการทำงานของตัวแทนที่มีเหตุผลสามารถสรุปได้เป็นลำดับดังนี้

1. รับข้อมูลจากสภาพแวดล้อมผ่านตัวรับข้อมูล (Sensors)
2. วิเคราะห์สถานะปัจจุบันของสิ่งแวดล้อม
3. ประเมินทางเลือกที่เป็นไปได้
4. เลือกการกระทำที่คาดว่าจะให้ผลลัพธ์ที่ดีที่สุด
5. ส่งคำสั่งผ่านตัวกระทำ (Actuators)
6. รับข้อมูลใหม่จากสภาพแวดล้อมและทำซ้ำกระบวนการ

วงจรดังกล่าวเรียกว่า วงจร รับรู้ - คิด - ปฏิบัติ (Sense-Think-Act Cycle) ซึ่งถือเป็นพื้นฐานสำคัญของระบบปัญญาประดิษฐ์และจะถูกนำไปประยุกต์ใช้ในขั้นตอนวิธีการค้นหา การวางแผน การเรียนรู้ของเครื่อง และหุ่นยนต์อัจฉริยะในบทต่อ ๆ ไป

2.2 การวัดประสิทธิภาพของตัวแทน

การออกแบบระบบปัญญาประดิษฐ์ไม่ได้พิจารณาเพียงว่าระบบสามารถทำงานได้หรือไม่ แต่ยังต้องพิจารณาว่าระบบสามารถทำงานได้ดีเพียงใดเมื่อเทียบกับเป้าหมายที่กำหนดไว้ การวัดความสามารถดังกล่าวเรียกว่า การวัดประสิทธิภาพ (Performance Measure) ซึ่งถือเป็นองค์ประกอบสำคัญของการออกแบบตัวแทนปัญญาประดิษฐ์

Russell และ Norvig (2021) กล่าวว่าตัวแทนที่มีเหตุผล (Rational Agent) จะเลือกการกระทำที่คาดว่าจะเพิ่มค่าประสิทธิภาพให้มากที่สุด ดังนั้นการกำหนดเกณฑ์การวัดประสิทธิภาพที่เหมาะสมจึงมีความสำคัญเนื่องจากตัวแทนจะใช้เกณฑ์ดังกล่าวเป็นเป้าหมายในการตัดสินใจ

การวัดประสิทธิภาพ หมายถึง เกณฑ์ที่ใช้ประเมินว่าตัวแทนสามารถบรรลุเป้าหมายได้ดีเพียงใดภายใต้สภาพแวดล้อมที่กำหนด

กล่าวอีกนัยหนึ่ง Performance Measure เป็นคำตอบของคำถามที่ว่า

"เราจะทราบได้อย่างไรว่าตัวแทนทำงานได้ดี?"

ทั้งนี้ เกณฑ์การวัดประสิทธิภาพอาจแตกต่างกันไปตามลักษณะของปัญหา เช่น

- ระบบนำทางรถยนต์ให้ความสำคัญกับความปลอดภัยและระยะเวลาในการเดินทาง
- ระบบวินิจฉัยโรคให้ความสำคัญกับความถูกต้องของผลลัพธ์
- หุ่นยนต์ดูดฝุ่นให้ความสำคัญกับความสะอาดและการใช้พลังงานอย่างมีประสิทธิภาพ

ดังนั้น การกำหนด Performance Measure ที่เหมาะสมจะส่งผลโดยตรงต่อพฤติกรรมของตัวแทน

• 2.2.1 ความถูกต้องของผลลัพธ์ (Correctness or Goal Achievement)

ความถูกต้องของผลลัพธ์เป็นเกณฑ์พื้นฐานที่ใช้ประเมินว่าตัวแทนสามารถบรรลุเป้าหมายที่กำหนดไว้ได้หรือไม่

ตัวอย่างเช่น

- ระบบวินิจฉัยโรคสามารถระบุโรคได้ถูกต้อง
- โปรแกรมเล่นหมากรุกสามารถเอาชนะคู่แข่งได้
- หุ่นยนต์สามารถเดินทางไปยังตำแหน่งเป้าหมายได้สำเร็จ

ในหลายกรณี ความถูกต้องของผลลัพธ์ถือเป็นปัจจัยที่สำคัญที่สุดของระบบปัญญาประดิษฐ์

• 2.2.2 ประสิทธิภาพในการทำงาน (Efficiency)

ประสิทธิภาพในการทำงาน หมายถึง ความสามารถในการให้ผลลัพธ์ภายในระยะเวลาที่เหมาะสม

ตัวอย่างเช่น

- ระบบแนะนำสินค้าควรแสดงผลลัพท์ภายในเวลาไม่กี่วินาที
- รถยนต์ขับเคลื่อนอัตโนมัติต้องตัดสินใจได้อย่างรวดเร็ว
- โปรแกรมตรวจจับวัตถุในภาพต้องทำงานแบบเวลาจริง (Real-time)

หากระบบมีความถูกต้องสูง แต่ใช้เวลาในการประมวลผลนานเกินไป อาจไม่สามารถนำไปใช้งานจริงได้

• 2.2.3 การใช้ทรัพยากร (Resource Usage)

ระบบปัญญาประดิษฐ์แต่ละระบบจำเป็นต้องใช้ทรัพยากรในการประมวลผล เช่น

- หน่วยความจำ (Memory)
- หน่วยประมวลผล (CPU)
- หน่วยประมวลผลกราฟิก (GPU)
- พลังงานไฟฟ้า
- พื้นที่จัดเก็บข้อมูล

ตัวอย่างเช่น หุ่นยนต์ดูดฝุ่นที่ใช้พลังงานน้อยจะสามารถทำงานได้ต่อเนื่องเป็นเวลานานกว่า ในขณะที่ระบบปัญญาประดิษฐ์ที่ใช้หน่วยความจำจำนวนมากอาจไม่เหมาะกับอุปกรณ์เคลื่อนที่ ดังนั้น การใช้ทรัพยากรอย่างมีประสิทธิภาพจึงเป็นปัจจัยสำคัญในการออกแบบระบบ

• 2.2.4 ความสามารถในการปรับตัว (Adaptability)

สภาพแวดล้อมในโลกจริงมีการเปลี่ยนแปลงอยู่ตลอดเวลา ตัวแทนที่ดีควรสามารถปรับตัวให้เข้ากับสถานการณ์ใหม่ได้ ตัวอย่างเช่น

- ระบบแนะนำภาพยนตร์สามารถเรียนรู้ความชอบของผู้ใช้ที่เปลี่ยนแปลงไป
- โปรแกรมเล่นเกมสามารถรับมือกับกลยุทธ์ใหม่ของคู่แข่งได้
- รถยนต์ขับเคลื่อนอัตโนมัติสามารถปรับเส้นทางเมื่อพบอุบัติเหตุหรือการจราจรติดขัด

ความสามารถในการปรับตัวช่วยให้ระบบสามารถทำงานได้อย่างมีประสิทธิภาพภายใต้สภาพแวดล้อมที่เปลี่ยนแปลงตลอดเวลา

• 2.2.5 ความทนทานต่อความผิดพลาด (Robustness)

ในโลกแห่งความเป็นจริง ข้อมูลที่ได้รับอาจมีความไม่สมบูรณ์หรือมีสัญญาณรบกวน ตัวแทนที่ดีควรสามารถทำงานได้แม้ว่าจะเกิดสถานการณ์ที่ไม่คาดคิดขึ้น ตัวอย่างเช่น

- รถยนต์ขับเคลื่อนอัตโนมัติยังคงสามารถทำงานได้แม้ว่ากล้องบางตัวจะทำงานผิดปกติ
- ระบบรู้จำเสียงสามารถทำงานได้แม้อยู่ในสภาพแวดล้อมที่มีเสียงรบกวน
- หุ่นยนต์สามารถหลีกเลี่ยงสิ่งกีดขวางได้แม้ว่าข้อมูลจากเซนเซอร์บางส่วนจะสูญหาย

คุณสมบัตินี้มีความสำคัญอย่างมากในระบบที่เกี่ยวข้องกับความปลอดภัย เช่น การแพทย์ หุ่นยนต์ หรือยานยนต์อัตโนมัติ

• 2.2.6 ความสามารถในการเรียนรู้ (Learning Ability)

ระบบปัญญาประดิษฐ์สมัยใหม่จำนวนมากสามารถปรับปรุงประสิทธิภาพของตนเองได้จากประสบการณ์ที่ผ่านมา ตัวอย่างเช่น

- ระบบแนะนำสินค้าสามารถเรียนรู้พฤติกรรมของผู้ใช้
- โปรแกรมเล่นหมากรุกสามารถพัฒนากลยุทธ์การเล่นจากการแข่งขันที่ผ่านมา
- โมเดลการเรียนรู้เชิงลึกสามารถปรับปรุงความแม่นยำจากข้อมูลฝึกสอนเพิ่มเติม

ความสามารถในการเรียนรู้ถือเป็นลักษณะสำคัญของปัญญาประดิษฐ์สมัยใหม่ และเป็นพื้นฐานของศาสตร์ด้านการเรียนรู้ของเครื่อง (Machine Learning)

• 2.2.7 ตัวอย่างการวัดประสิทธิภาพของตัวแทน

สมมติว่าต้องการออกแบบรถยนต์ขับเคลื่อนอัตโนมัติ ตัวอย่างเกณฑ์การวัดประสิทธิภาพอาจประกอบด้วย

1. เดินทางถึงจุดหมายได้สำเร็จ
2. ไม่เกิดอุบัติเหตุ
3. ใช้เวลาในการเดินทางน้อยที่สุด
4. ประหยัดพลังงานหรือเชื้อเพลิง
5. ปฏิบัติตามกฎหมายจราจร
6. สามารถตอบสนองต่อเหตุการณ์ฉุกเฉินได้อย่างเหมาะสม

จะเห็นได้ว่า ตัวแทนไม่ได้มุ่งเน้นเพียงการเดินทางถึงจุดหมายเท่านั้น แต่ยังต้องคำนึงถึงปัจจัยอื่น ๆ ร่วมด้วย

ตารางที่ 2.4 ตัวอย่างเกณฑ์การวัดประสิทธิภาพของตัวแทน

ตัวแทน	เกณฑ์การวัดประสิทธิภาพ
รถยนต์ขับเคลื่อนอัตโนมัติ	ความปลอดภัย เวลาในการเดินทาง การใช้พลังงาน
ระบบวินิจฉัยโรค	ความถูกต้อง ความรวดเร็ว ความน่าเชื่อถือ
โปรแกรมเล่นหมากรุก	จำนวนครั้งที่ชนะ ระยะเวลาในการตัดสินใจ
หุ่นยนต์ดูดฝุ่น	พื้นที่ที่ทำความสะอาดได้ การใช้พลังงาน
ระบบแนะนำสินค้า	ความแม่นยำ ความพึงพอใจของผู้ใช้

จากตัวอย่างข้างต้นจะเห็นได้ว่า การกำหนด Performance Measure ที่แตกต่างกันอาจส่งผลให้ตัวแทนมีพฤติกรรมที่แตกต่างกัน แม้ว่าจะเป็นระบบประเภทเดียวกันก็ตาม ดังนั้น การกำหนดเกณฑ์การวัดประสิทธิภาพที่เหมาะสมจึงเป็นขั้นตอนสำคัญในการออกแบบระบบปัญญาประดิษฐ์

กล่าวโดยสรุป ตัวแทนที่มีเหตุผล (Rational Agent) จะเลือกการกระทำที่ทำให้ค่าประสิทธิภาพ (Performance Measure) สูงที่สุด ดังนั้น Performance Measure จึงเปรียบเสมือนเป้าหมายหรือเกณฑ์ในการตัดสินใจของตัวแทน และถือเป็นองค์ประกอบสำคัญที่กำหนดพฤติกรรมของระบบปัญญาประดิษฐ์

2.3 ตัวอย่างตัวแทนปัญญาประดิษฐ์

ตัวแทนปัญญาประดิษฐ์ (Intelligent Agent) สามารถพบได้ในชีวิตประจำวันหลากหลายรูปแบบ ตั้งแต่ระบบซอฟต์แวร์ที่ทำงานอยู่บนคอมพิวเตอร์ ไปจนถึงหุ่นยนต์และยานยนต์อัตโนมัติ แม้ว่ารูปแบบการทำงานของตัวแทนแต่ละชนิดจะแตกต่างกัน แต่ทุกระบบล้วนมีองค์ประกอบพื้นฐานเหมือนกัน ได้แก่ สภาพแวดล้อม (Environment) ตัวรับข้อมูล (Sensors) ตัวกระทำ (Actuators) และเกณฑ์การวัดประสิทธิภาพ (Performance Measure)

การศึกษาองค์ประกอบเหล่านี้ช่วยให้สามารถออกแบบตัวแทนที่มีเหตุผล (Rational Agent) ได้อย่างเหมาะสมกับลักษณะของปัญหา

2.3.1 ตัวแทนคนขับรถแท็กซี่อัตโนมัติ (Taxi Driver Agent)

ตัวแทนคนขับรถแท็กซี่อัตโนมัติเป็นตัวอย่างคลาสสิกที่ถูกกล่าวถึงในหนังสือ Artificial Intelligence: A Modern Approach โดยมีวัตถุประสงค์เพื่อรับส่งผู้โดยสารไปยังจุดหมายปลายทางอย่างปลอดภัยและมีประสิทธิภาพ

สภาพแวดล้อมของตัวแทนประกอบด้วย ถนน การจราจร สภาพอากาศ ผู้ใช้ถนนคันอื่น รวมถึงกฎจราจรต่าง ๆ

ตัวรับข้อมูลของระบบอาจประกอบด้วย

- กล้อง (Camera)
- GPS
- LiDAR
- เรดาร์ (Radar)
- เซนเซอร์วัดระยะ
- ข้อมูลจากแผนที่

ส่วนตัวกระทำของระบบ ได้แก่

- พวงมาลัย
- คันเร่ง

- ระบบเบรก
- ไฟเลี้ยว
- ระบบสื่อสารกับผู้โดยสาร
สำหรับการวัดประสิทธิภาพของตัวแทนประเภทนี้ ได้แก่
- เดินทางถึงจุดหมายได้สำเร็จ
- ไม่เกิดอุบัติเหตุ
- ใช้เวลาในการเดินทางน้อยที่สุด
- ประหยัดพลังงานหรือเชื้อเพลิง
- ปฏิบัติตามกฎหมายจราจร

2.3.2 ระบบวินิจฉัยโรค (Medical Diagnosis System)

ระบบวินิจฉัยโรคเป็นตัวแทนปัญญาประดิษฐ์ที่ถูกนำมาใช้ช่วยแพทย์ในการวิเคราะห์โรคและให้คำแนะนำเบื้องต้นแก่ผู้ป่วย

สภาพแวดล้อมของระบบประกอบด้วย

- ข้อมูลผู้ป่วย
- ประวัติการรักษา
- ผลการตรวจทางห้องปฏิบัติการ
- ภาพเอกซเรย์
- ฐานข้อมูลทางการแพทย์

ตัวรับข้อมูลของระบบ ได้แก่

- แบบสอบถามผู้ป่วย
- ผลตรวจเลือด
- ภาพเอกซเรย์
- เครื่องวัดสัญญาณชีพ

ตัวกระทำของระบบ ได้แก่

- รายงานผลการวินิจฉัย
- คำแนะนำในการรักษา
- การแจ้งเตือนแพทย์

เกณฑ์การวัดประสิทธิภาพของระบบ ได้แก่

- ความถูกต้องในการวินิจฉัย
- ความรวดเร็วในการประมวลผล

- ความน่าเชื่อถือของผลลัพธ์
- การลดความผิดพลาดในการรักษา

2.3.3 หุ่นยนต์ดูดฝุ่นอัตโนมัติ (Vacuum Cleaning Robot)

หุ่นยนต์ดูดฝุ่นเป็นตัวอย่างของตัวแทนที่ทำงานในสภาพแวดล้อมทางกายภาพ โดยมีหน้าที่ทำความสะอาดพื้นที่ภายในบ้าน

สภาพแวดล้อมของระบบประกอบด้วย

- ห้องต่าง ๆ ภายในบ้าน
- สิ่งกีดขวาง
- เฟอร์นิเจอร์
- พื้นผิวชนิดต่าง ๆ

ตัวรับข้อมูลของระบบ ได้แก่

- เซนเซอร์ตรวจจับสิ่งกีดขวาง
- กล้อง
- เซนเซอร์อินฟราเรด
- เซนเซอร์วัดระยะ

ตัวกระทำของระบบ ได้แก่

- มอเตอร์ขับเคลื่อน
 - ล้อ
 - ระบบดูดฝุ่น
- เกณฑ์การวัดประสิทธิภาพ ได้แก่
- พื้นที่ทำความสะอาดได้
 - ระยะเวลาในการทำงาน
 - การใช้พลังงาน
 - ความสามารถในการหลีกเลี่ยงสิ่งกีดขวาง

2.3.4 โปรแกรมเล่นหมากรุก (Chess Playing Program)

โปรแกรมเล่นหมากรุกเป็นตัวแทนที่ทำงานอยู่ในสภาพแวดล้อมแบบเสมือน โดยมีเป้าหมายเพื่อเอาชนะคู่แข่ง

สภาพแวดล้อมของระบบประกอบด้วย

- กระดานหมากรุก
- ตัวหมากทั้งสองฝ่าย

- กฎการเล่นหมากรุก
ตัวรับข้อมูลของระบบ ได้แก่
- ตำแหน่งของหมากทุกตัวบนกระดาน
- การเดินหมากของฝ่ายตรงข้าม
ตัวกระทำของระบบ ได้แก่
- การเลือกตำแหน่งเดินหมากถัดไป
เกณฑ์การวัดประสิทธิภาพ ได้แก่
- จำนวนครั้งที่ชนะการแข่งขัน
- ระยะเวลาในการตัดสินใจ
- ความสามารถในการคาดการณ์การเดินของคู่แข่ง

2.3.5 ระบบแนะนำสินค้า (Recommendation System)

ระบบแนะนำสินค้าเป็นตัวแทนที่พบได้ทั่วไปในแพลตฟอร์มอีคอมเมิร์ซและบริการสตรีมมิงต่าง ๆ

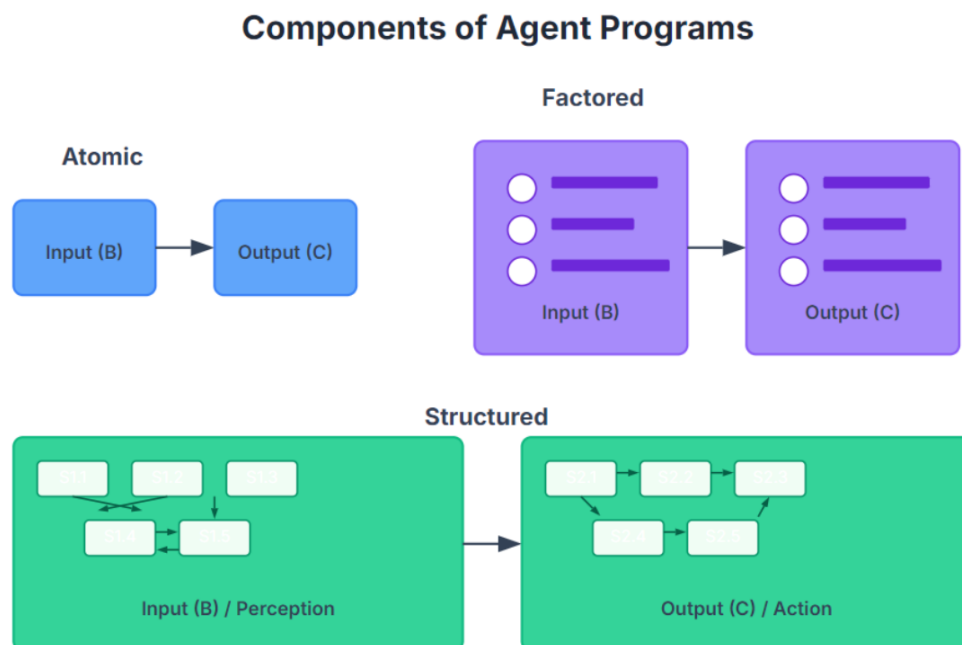
สภาพแวดล้อมของระบบประกอบด้วย

- ข้อมูลผู้ใช้
- ประวัติการซื้อสินค้า
- พฤติกรรมการใช้งาน
- ข้อมูลสินค้า
ตัวรับข้อมูลของระบบ ได้แก่
- การคลิกสินค้า
- ประวัติการค้นหา
- การให้คะแนนสินค้า
- ประวัติการสั่งซื้อ
ตัวกระทำของระบบ ได้แก่
- การแนะนำสินค้า
- การแสดงโฆษณาเฉพาะบุคคล
เกณฑ์การวัดประสิทธิภาพ ได้แก่
- ความแม่นยำในการแนะนำ
- ความพึงพอใจของผู้ใช้
- อัตราการคลิก (Click Through Rate)
- ยอดขายที่เพิ่มขึ้น

จากตัวอย่างข้างต้นจะเห็นได้ว่า แม้ว่าตัวแทนปัญญาประดิษฐ์แต่ละชนิดจะมีวัตถุประสงค์และสภาพแวดล้อมที่ต่างกัน แต่ทุกระบบล้วนประกอบด้วยองค์ประกอบพื้นฐานเหมือนกัน ได้แก่ สภาพแวดล้อม ตัวรับข้อมูล ตัวกระทำ และเกณฑ์การวัดประสิทธิภาพ ซึ่งองค์ประกอบเหล่านี้ถือเป็นพื้นฐานสำคัญของการออกแบบตัวแทนที่มีเหตุผล

2.4 ส่วนประกอบของตัวแทน

ในการศึกษาปัญญาประดิษฐ์ เราไม่ได้สนใจเพียงว่าตัวแทนสามารถรับข้อมูลและแสดงการกระทำได้อย่างไรเท่านั้น แต่ยังต้องพิจารณาถึงระดับของการแทนความรู้ (Representation) และโครงสร้างภายในของตัวแทนด้วย เนื่องจากระดับของการแทนข้อมูลส่งผลโดยตรงต่อความสามารถในการวิเคราะห์ ออกแบบ และพัฒนาระบบปัญญาประดิษฐ์



รูปที่ 2.2 องค์ประกอบของตัวแทนทั้ง 3 ประเภท

Russell และ Norvig ได้แบ่งการแทนปัญหาและองค์ประกอบของตัวแทนออกเป็นสามระดับดังรูปที่ 2.2 ได้แก่

1. แบบอะตอม (Atomic Representation)
2. แบบแยกส่วน (Factored Representation)
3. แบบมีโครงสร้าง (Structured Representation)

ระดับของการแทนข้อมูลทั้งสามรูปแบบแสดงให้เห็นถึงความแตกต่างของปริมาณข้อมูลและความสัมพันธ์ระหว่างองค์ประกอบต่าง ๆ ที่ตัวแทนสามารถรับรู้และนำมาใช้ในการตัดสินใจ

• 2.4.1 การแทนแบบอะตอม (Atomic Representation)

การแทนแบบอะตอม (Atomic Representation) เป็นรูปแบบที่ง่ายที่สุด โดยมองสถานะหรือข้อมูลทั้งหมดเป็นเพียงหน่วยเดียว (Single Entity) โดยไม่สนใจรายละเอียดหรือโครงสร้างภายใน

ในมุมมองนี้ ตัวแทนสามารถเปรียบเสมือนกล่องดำ (Black Box) กล่าวคือ เราทราบเพียงข้อมูลนำเข้าและผลลัพธ์ที่ได้ แต่ไม่ทราบรายละเอียดภายในของกระบวนการตัดสินใจ

ตัวอย่างเช่น

ในการเล่นหมากรุก หากพิจารณากระดานหมากรุกทั้งกระดานเป็นเพียงสถานะเดียว โดยไม่สนใจตำแหน่งของหมากแต่ละตัว การแทนข้อมูลลักษณะนี้ถือเป็นการแทนแบบอะตอม

ลักษณะสำคัญของการแทนแบบอะตอม ได้แก่

- มองข้อมูลเป็นหน่วยเดียว
- ไม่สนใจองค์ประกอบย่อยภายใน
- มีความเรียบง่าย
- เหมาะสำหรับปัญหาที่ไม่ซับซ้อน

ข้อดีของการแทนแบบอะตอมคือมีความง่ายต่อการออกแบบและการประมวลผล แต่ข้อจำกัดคือไม่สามารถอธิบายความสัมพันธ์ภายในของข้อมูลได้

• 2.4.2 การแทนแบบแยกส่วน (Factored Representation)

การแทนแบบแยกส่วน (Factored Representation) เป็นการแบ่งสถานะออกเป็นองค์ประกอบย่อยหลายส่วน (Variables) ทำให้สามารถอธิบายรายละเอียดของข้อมูลได้มากขึ้น

ตัวอย่างเช่น

ข้อมูลของนักศึกษาหนึ่งคนอาจประกอบด้วย

- รหัสนักศึกษา
- ชื่อ
- อายุ
- สาขาวิชา
- เกรดเฉลี่ย

ดังนั้น สถานะหนึ่งสถานะจะไม่ใช้เพียงหน่วยเดียวอีกต่อไป แต่ประกอบด้วยตัวแปรหลายตัว

ตัวอย่างการแทนข้อมูลในรูปแบบนี้สามารถเขียนได้ดังนี้

Student = (ID, Name, Age, Major, GPA)

ในการเล่นหมากรุก การแทนแบบแยกส่วนอาจพิจารณาตำแหน่งของหมากแต่ละตัวบนกระดาน ทำให้สามารถอธิบายสถานะของเกมได้ละเอียดกว่าการแทนแบบอะตอม

ลักษณะสำคัญของการแทนแบบแยกส่วน ได้แก่

- ข้อมูลประกอบด้วยตัวแปรหลายตัว
- สามารถอธิบายรายละเอียดของสถานะได้มากขึ้น
- ยังไม่แสดงความสัมพันธ์ระหว่างตัวแปรอย่างชัดเจน
- นิยมใช้ในระบบฐานข้อมูลและปัญญาประดิษฐ์แบบดั้งเดิม

การแทนแบบแยกส่วนเป็นรูปแบบที่พบได้บ่อยในงานด้านการเรียนรู้ของเครื่อง (Machine Learning) และระบบผู้เชี่ยวชาญ (Expert System)

• 2.4.3 การแทนแบบมีโครงสร้าง (Structured Representation)

การแทนแบบมีโครงสร้าง (Structured Representation) เป็นรูปแบบที่มีความซับซ้อนมากที่สุด โดยนอกจากจะทราบองค์ประกอบย่อยแล้ว ยังทราบความสัมพันธ์ระหว่างองค์ประกอบต่าง ๆ อีกด้วย

ตัวอย่างเช่น

ข้อมูลภายในครอบครัวหนึ่งอาจประกอบด้วย

- นายสมชาย เป็นบิดาของเด็กชายเอ
- นางสาวหญิง เป็นมารดาของเด็กชายเอ

ในกรณีนี้ นอกจากจะทราบข้อมูลของบุคคลแต่ละคนแล้ว ยังทราบความสัมพันธ์ระหว่างบุคคลอีกด้วย การแทนข้อมูลลักษณะนี้สามารถเขียนในรูปของกราฟหรือฟังก์ชันได้ เช่น Father (Somchai, A) Mother (Somying, A)

ตัวอย่างอื่นของการแทนแบบมีโครงสร้าง ได้แก่

- Semantic Network
- Knowledge Graph
- Bayesian Network
- Ontology
- Relational Database

ลักษณะสำคัญของการแทนแบบมีโครงสร้าง ได้แก่

- สามารถอธิบายความสัมพันธ์ระหว่างข้อมูลได้
- มีความยืดหยุ่นสูง
- เหมาะสำหรับปัญหาที่มีความซับซ้อน
- สามารถนำไปใช้ในการให้เหตุผล (Reasoning) ได้

ปัจจุบันระบบปัญญาประดิษฐ์สมัยใหม่จำนวนมาก เช่น Large Language Model (LLM) และระบบ Knowledge Graph ต่างอาศัยแนวคิดของการแทนข้อมูลแบบมีโครงสร้างร่วมกับเทคนิคการเรียนรู้ของเครื่อง เพื่อเพิ่มความสามารถในการวิเคราะห์และให้เหตุผล

- 2.4.4 การเปรียบเทียบรูปแบบการแทนข้อมูลทั้งสามประเภท

ตารางที่ 2.5 เปรียบเทียบการแทนข้อมูลทั้งสามรูปแบบ

คุณลักษณะ	Atomic	Factored	Structured
จำนวนองค์ประกอบภายใน	ไม่พิจารณา	หลายตัวแปร	หลายตัวแปรและ ความสัมพันธ์
ความซับซ้อน	ต่ำ	ปานกลาง	สูง
การแสดงความสัมพันธ์ระหว่าง ข้อมูล	ไม่มี	จำกัด	มี
การให้เหตุผลเชิงลึก	ไม่สามารถทำได้	ทำได้บางส่วน	ทำได้ดี
ตัวอย่าง	สถานะของเกมทั้งหมด	ตารางข้อมูล	Knowledge Graph

จะเห็นได้ว่าการแทนข้อมูลทั้งสามรูปแบบมีระดับความซับซ้อนที่แตกต่างกัน โดยการแทนแบบอะตอมมีความเรียบง่ายที่สุด ในขณะที่การแทนแบบมีโครงสร้างสามารถอธิบายความสัมพันธ์ระหว่างข้อมูลได้อย่างละเอียดมากที่สุด

การเลือกใช้รูปแบบการแทนข้อมูลที่เหมาะสมถือเป็นขั้นตอนสำคัญในการออกแบบระบบปัญญาประดิษฐ์ เนื่องจากมีผลต่อประสิทธิภาพในการค้นหา การให้เหตุผล และความสามารถในการเรียนรู้ของระบบ

2.5 การแก้ปัญหาในปัญญาประดิษฐ์

ความสามารถในการแก้ปัญหา (Problem Solving) ถือเป็นหนึ่งในเป้าหมายสำคัญของปัญญาประดิษฐ์ เนื่องจากปัญหาต่าง ๆ ที่เกิดขึ้นในโลกแห่งความเป็นจริงล้วนต้องอาศัยการวิเคราะห์ การตัดสินใจ และการค้นหาวิธีการที่เหมาะสมที่สุดเพื่อให้บรรลุเป้าหมายที่ต้องการ

มนุษย์ใช้กระบวนการแก้ปัญหาอยู่ตลอดเวลา เช่น การวางแผนเส้นทางการเดินทาง การแก้โจทย์คณิตศาสตร์ การเล่นเกมหมากรุก หรือการวางแผนทางธุรกิจ ในทำนองเดียวกัน ระบบปัญญาประดิษฐ์ก็ต้องสามารถเลือกวิธีการแก้ปัญหาที่เหมาะสมเพื่อให้ได้ผลลัพธ์ที่ดีที่สุด

ในทางปัญญาประดิษฐ์ การแก้ปัญหามักถูกนิยามให้อยู่ในรูปของการค้นหา (Search) ภายในปริภูมิสถานะ (State Space) โดยมีวัตถุประสงค์เพื่อค้นหาลำดับของการกระทำ (Action Sequence) ที่สามารถนำระบบจากสถานะเริ่มต้นไปยังสถานะเป้าหมาย

2.5.1 ตัวแทนแก้ปัญหา (Problem-Solving Agent)

ตัวแทนแก้ปัญหา (Problem-Solving Agent) เป็นตัวแทนที่เลือกการกระทำโดยอาศัยการค้นหาเส้นทางจากสถานะปัจจุบันไปยังสถานะเป้าหมาย

กระบวนการทำงานของตัวแทนแก้ปัญหาสามารถแบ่งออกได้เป็น 4 ขั้นตอน ดังนี้

1. รับรู้สภาพแวดล้อม (Perception)

ตัวแทนรับข้อมูลจากสภาพแวดล้อมผ่านตัวรับข้อมูล (Sensors) เพื่อสร้างภาพแทนของสถานะปัจจุบัน

2. กำหนดเป้าหมาย (Goal Formulation)

กำหนดวัตถุประสงค์หรือสถานะที่ต้องการให้เกิดขึ้น

3. กำหนดปัญหา (Problem Formulation)

แปลงปัญหาให้อยู่ในรูปแบบที่สามารถนำไปวิเคราะห์และประมวลผลได้

4. ค้นหาคำตอบ (Search)

เลือกขั้นตอนวิธีการค้นหาที่เหมาะสมเพื่อค้นหาเส้นทางหรือวิธีการแก้ปัญหา

กระบวนการดังกล่าวสามารถสรุปได้ดังนี้

Perception → Goal Formulation → Problem Formulation → Search → Solution

จะเห็นได้ว่าการค้นหา (Search) เป็นเพียงส่วนหนึ่งของกระบวนการแก้ปัญหาเท่านั้น ก่อนที่จะเริ่มค้นหา จำเป็นต้องกำหนดเป้าหมายและนิยามปัญหาให้ชัดเจนเสียก่อน

2.5.2 ขั้นตอนการแก้ปัญหา

การแก้ปัญหาในงานด้านปัญญาประดิษฐ์สามารถแบ่งออกเป็น 6 ขั้นตอนหลัก ดังนี้

1) ทำความเข้าใจปัญหา (Define the Problem)

ขั้นตอนแรกคือการวิเคราะห์ลักษณะของปัญหา กำหนดข้อมูลนำเข้า ผลลัพธ์ที่ต้องการ และข้อจำกัดต่าง ๆ

ตัวอย่างเช่น

หากต้องการสร้างระบบนำทางรถยนต์ จะต้องกำหนดให้ชัดเจนว่า

- จุดเริ่มต้นอยู่ที่ใด
- จุดหมายปลายทางอยู่ที่ใด
- มีถนนหรือเส้นทางใดที่สามารถเลือกใช้ได้บ้าง
- ต้องการลดเวลาในการเดินทางหรือระยะทางรวม

การกำหนดปัญหาที่ไม่ชัดเจนอาจทำให้ไม่สามารถค้นหาคำตอบที่เหมาะสมได้

2) เลือกวิธีการแก้ปัญหา (Choose a Search Strategy)

เมื่อเข้าใจปัญหาแล้ว ขั้นตอนถัดไปคือการเลือกขั้นตอนวิธี (Algorithm) ที่เหมาะสม ตัวอย่างเช่น

- Breadth-First Search
- Depth-First Search
- Greedy Search
- A* Search
- Minimax Algorithm

ขั้นตอนวิธีแต่ละชนิดมีข้อดีและข้อจำกัดแตกต่างกัน ดังนั้นจึงต้องเลือกให้เหมาะสมกับลักษณะของ

ปัญหา

3) นำขั้นตอนวิธีไปพัฒนาโปรแกรม (Implement Search Algorithm)

หลังจากเลือกขั้นตอนวิธีที่เหมาะสมแล้ว จึงนำมาพัฒนาเป็นโปรแกรมคอมพิวเตอร์ ขั้นตอนนี้ประกอบด้วย

- การออกแบบโครงสร้างข้อมูล
- การเขียนโปรแกรม
- การทดสอบความถูกต้องเบื้องต้น

4) ปรับปรุงและจัดการกรณีพิเศษ (Handle Edge Cases and Optimization)

ในการทำงานจริง อาจเกิดสถานการณ์ที่ไม่คาดคิดขึ้น เช่น

- การวนซ้ำไม่สิ้นสุด (Infinite Loop)
- การใช้หน่วยความจำมากเกินไป
- การค้นหาใช้เวลานานเกินไป
- ข้อมูลนำเข้าผิดพลาด

ดังนั้นจึงจำเป็นต้องปรับปรุงระบบให้สามารถรองรับสถานการณ์เหล่านี้ได้

5) การตรวจสอบและทวนสอบ (Verification and Validation)

Verification เป็นการตรวจสอบว่าโปรแกรมทำงานถูกต้องตามที่ออกแบบไว้หรือไม่

Validation เป็นการตรวจสอบว่าโปรแกรมสามารถแก้ปัญหาลงมือได้ตรงตามความต้องการหรือไม่

ตัวอย่างเช่น

ระบบนำทางอาจทำงานได้ถูกต้องตามอัลกอริทึมที่ออกแบบไว้ แต่เส้นทางที่แนะนำอาจไม่เหมาะสม

กับสภาพการจราจรจริง

ดังนั้นทั้ง Verification และ Validation จึงมีความสำคัญต่อการพัฒนาระบบปัญญาประดิษฐ์

6) การปรับปรุงอย่างต่อเนื่อง (Iterate and Improve)

ระบบปัญญาประดิษฐ์จำนวนมากจำเป็นต้องมีการปรับปรุงประสิทธิภาพอย่างต่อเนื่อง

ตัวอย่างเช่น

- เพิ่มความเร็วในการค้นหา
- ลดการใช้หน่วยความจำ
- เพิ่มความแม่นยำของผลลัพธ์
- ปรับปรุงโมเดลให้เรียนรู้ได้ดีขึ้น

วงจรการพัฒนาในลักษณะนี้เรียกว่า Iterative Improvement

2.5.3 ปัญหาของเล่นและปัญหาในโลกจริง

ในช่วงเริ่มต้นของการวิจัยปัญญาประดิษฐ์ นักวิจัยมักนิยมศึกษาปัญหาของเล่น (Toy Problems) มากกว่าปัญหาในโลกจริง

ปัญหาของเล่น หมายถึง ปัญหาที่มีขนาดเล็ก มีโครงสร้างชัดเจน และสามารถควบคุมเงื่อนไขต่าง ๆ ได้ง่าย

ตัวอย่างของปัญหาของเล่น ได้แก่

- ปัญหา 8 ตัวเลข (8-Puzzle)
- ปัญหา 8 ราชีนี (8-Queens Problem)
- ปัญหาการเดินทางของเซลแมน (Travelling Salesman Problem)
- เกมหมากรุก

การใช้ปัญหาของเล่นมีข้อดีหลายประการ ได้แก่

1. ช่วยสร้างความเข้าใจในหลักการพื้นฐานของปัญญาประดิษฐ์
2. สามารถทดสอบขั้นตอนวิธีใหม่ได้ง่าย
3. ใช้ทรัพยากรในการคำนวณไม่มาก
4. สามารถเปรียบเทียบประสิทธิภาพของขั้นตอนวิธีต่าง ๆ ได้อย่างชัดเจน

เมื่อแนวคิดและขั้นตอนวิธีได้รับการพัฒนาและพิสูจน์ประสิทธิภาพแล้ว จึงสามารถนำไปประยุกต์ใช้กับปัญหาในโลกแห่งความเป็นจริงที่มีความซับซ้อนมากขึ้น

ดังนั้น การศึกษาปัญหาของเล่นจึงถือเป็นจุดเริ่มต้นสำคัญของการพัฒนาปัญญาประดิษฐ์ และเป็นพื้นฐานของขั้นตอนวิธีการค้นหาต่าง ๆ ที่จะกล่าวถึงในหัวข้อถัดไป

2.5.4 ความสำคัญของการแทนปัญหา

การแก้ปัญหาในปัญญาประดิษฐ์ไม่ได้ขึ้นอยู่กับขั้นตอนวิธีเพียงอย่างเดียว แต่ขึ้นอยู่กับวิธีการแทนปัญหา (Problem Representation) ด้วย การแทนปัญหาที่ดีจะช่วยให้

- ลดความซับซ้อนของปัญหา
- ทำให้สามารถเลือกขั้นตอนวิธีที่เหมาะสมได้

- เพิ่มประสิทธิภาพในการค้นหา
- ลดเวลาในการคำนวณ

ดังนั้นก่อนที่จะเริ่มค้นหาคำตอบ จำเป็นต้องแปลงปัญหาให้อยู่ในรูปแบบที่ชัดเจนและสามารถวิเคราะห์ได้ ซึ่งเรียกว่า ปัญหาที่นิยามไว้อย่างชัดเจน (Well-defined Problem)

2.6 การแทนปัญหาให้อยู่ในรูปมาตรฐาน

ก่อนที่จะระบบปัญญาประดิษฐ์จะสามารถค้นหาคำตอบของปัญหาได้ จำเป็นต้องแปลงปัญหาให้อยู่ในรูปแบบที่ชัดเจนและสามารถวิเคราะห์ได้เสียก่อน กระบวนการดังกล่าวเรียกว่า การแทนปัญหา (Problem Representation) หรือการนิยามปัญหาให้อยู่ในรูปแบบมาตรฐาน (Well-defined Problem)

การแทนปัญหาที่เหมาะสมถือเป็นขั้นตอนสำคัญของการแก้ปัญหา เนื่องจากขั้นตอนวิธีเดียวกันอาจให้ประสิทธิภาพที่แตกต่างกัน หากมีการแทนปัญหาที่แตกต่างกัน

Russell และ Norvig ได้กำหนดองค์ประกอบของปัญหาที่นิยามไว้อย่างชัดเจนไว้ 5 องค์ประกอบ ดังนี้

1. สถานะเริ่มต้น (Initial State)
2. การกระทำ (Actions)
3. แบบจำลองการเปลี่ยนสถานะ (Transition Model)
4. สถานะเป้าหมาย (Goal State)
5. ค่าใช้จ่ายของเส้นทาง (Path Cost)

องค์ประกอบทั้งห้าส่วนนี้เป็นพื้นฐานสำคัญของขั้นตอนวิธีการค้นหา (Search Algorithm) ในงานด้านปัญญาประดิษฐ์

2.6.1 สถานะเริ่มต้น (Initial State)

สถานะเริ่มต้น คือ สถานะที่ตัวแทนอยู่ก่อนเริ่มกระบวนการแก้ปัญหา หรือจุดเริ่มต้นของการค้นหา สถานะเริ่มต้นอาจเป็น

- ตำแหน่งเริ่มต้นของหุ่นยนต์
- การจัดเรียงตัวเลขเริ่มต้นของปัญหา 8 ตัวเลข
- เมืองต้นทางของปัญหาการเดินทางของเซลแมน

ตัวอย่างเช่น หากต้องการหาเส้นทางจากกรุงเทพมหานครไปยังเชียงใหม่

สถานะเริ่มต้น คือ กรุงเทพมหานคร

การกำหนดสถานะเริ่มต้นที่ถูกต้องจะช่วยให้กระบวนการค้นหาสามารถเริ่มต้นได้อย่างเหมาะสม

2.6.2 การกระทำ (Actions)

การกระทำ (Actions) หมายถึง ชุดของการกระทำทั้งหมดที่สามารถเกิดขึ้นได้ในแต่ละสถานะ การกระทำทำให้ตัวแทนสามารถเปลี่ยนจากสถานะหนึ่งไปสู่อีกสถานะหนึ่งได้ ตัวอย่างเช่น ในปัญหาเขาวงกต การกระทำที่เป็นไปได้ ได้แก่

- เดินขึ้น
- เดินลง
- เดินซ้าย
- เดินขวา

ในเกมหมากรุก การกระทำที่เป็นไปได้ คือ

- การเดินหมากแต่ละรูปแบบตามกฎหมายของเกม

ในปัญหา 8 ตัวเลข การกระทำ คือ

- เลื่อนช่องว่างขึ้น
- เลื่อนช่องว่างลง
- เลื่อนช่องว่างไปทางซ้าย
- เลื่อนช่องว่างไปทางขวา

ชุดของการกระทำเหล่านี้เรียกว่า Action Space

2.6.3 แบบจำลองการเปลี่ยนสถานะ (Transition Model)

แบบจำลองการเปลี่ยนสถานะ (Transition Model) เป็นกฎที่ใช้อธิบายว่า เมื่อมีการกระทำใด ๆ เกิดขึ้น ระบบจะเปลี่ยนจากสถานะปัจจุบันไปเป็นสถานะใหม่อย่างไร

สามารถเขียนในรูปฟังก์ชันได้ดังนี้

$$\text{Result}(s, a) \rightarrow s'$$

เมื่อ

s คือ สถานะปัจจุบัน (Current State)

a คือ การกระทำ (Action)

s' คือ สถานะใหม่ (Successor State)

ตัวอย่างเช่น ถ้าหุ่นยนต์อยู่ที่ตำแหน่ง (2,3) และเลือกการกระทำ Move Right สถานะใหม่จะกลายเป็น (3,3) ดังนั้น $\text{Result}((2,3), \text{Move Right}) = (3,3)$

แบบจำลองการเปลี่ยนสถานะมีบทบาทสำคัญในการสร้างต้นไม้การค้นหา (Search Tree) และกราฟสถานะ (State Graph)

2.6.4 สถานะเป้าหมาย (Goal State)

สถานะเป้าหมาย (Goal State) คือ สถานะที่ต้องการให้เกิดขึ้นเมื่อสิ้นสุดกระบวนการค้นหา กล่าวอีกนัยหนึ่งคือคำตอบของปัญหา ตัวอย่างเช่น

- ในปัญหาการหาเส้นทาง สถานะเป้าหมายอาจเป็น เชียงใหม่
- ในเกมหมากรุก สถานะเป้าหมายอาจเป็น Checkmate ฝ่ายตรงข้าม
- ในปัญหา 8 ตัวเลข สถานะเป้าหมายอาจเป็น

1 2 3

4 5 6

7 8 □

ระบบจะทำการค้นหาต่อไปจนกว่าจะพบสถานะที่ตรงกับเงื่อนไขเป้าหมาย

2.6.5 ค่าใช้จ่ายของเส้นทาง (Path Cost)

ในการแก้ปัญหา อาจมีหลายเส้นทางที่สามารถนำไปสู่สถานะเป้าหมายได้ ดังนั้นจึงจำเป็นต้องมีเกณฑ์สำหรับเปรียบเทียบว่าเส้นทางใดดีกว่า

ค่าใช้จ่ายของเส้นทาง (Path Cost) คือค่าที่ใช้วัดทรัพยากรที่ใช้ในการเดินทางจากสถานะเริ่มต้นไปยังสถานะเป้าหมาย ตัวอย่างของค่าใช้จ่าย ได้แก่

- ระยะทาง
- เวลา
- จำนวนก้าวที่เคลื่อนที่
- พลังงานที่ใช้
- ค่าใช้จ่ายทางการเงิน

ตัวอย่างเช่น

เส้นทางที่ 1

$A \rightarrow B \rightarrow D \rightarrow G$

มีค่าใช้จ่ายรวมเท่ากับ 20

เส้นทางที่ 2

$A \rightarrow C \rightarrow E \rightarrow G$

มีค่าใช้จ่ายรวมเท่ากับ 15

ดังนั้นเส้นทางที่สองถือว่าเป็นเส้นทางที่ดีกว่า หากต้องการลดค่าใช้จ่ายให้น้อยที่สุด

2.6.6 ตัวอย่างการแทนปัญหาในรูปแบบมาตรฐาน

พิจารณาปัญหาการเดินทางจากจังหวัดเชียงรายไปยังจังหวัดเชียงใหม่

- **สถานะเริ่มต้น** เชียงราย
- **การกระทำ** เลือกถนนที่สามารถเดินทางได้ไปยังจังหวัดใกล้เคียง
- **แบบจำลองการเปลี่ยนสถานะ** เมื่อเลือกเส้นทางหนึ่ง จะย้ายจากจังหวัดปัจจุบันไปยังจังหวัดถัดไป
- **สถานะเป้าหมาย** เชียงใหม่
- **ค่าใช้จ่ายของเส้นทาง** ระยะทางรวมที่ใช้ในการเดินทาง

จะเห็นได้ว่า การกำหนดองค์ประกอบทั้งห้าส่วนอย่างชัดเจน ช่วยให้ปัญหาสามารถถูกนำไปแก้ไขด้วยขั้นตอนวิธีการค้นหาต่าง ๆ ได้

2.6.7 ความสำคัญของการแทนปัญหา

การแทนปัญหาที่ดีช่วยให้

- ลดความซับซ้อนของปัญหา
- ทำให้สามารถเลือกขั้นตอนวิธีที่เหมาะสมได้
- ลดเวลาในการค้นหา
- ลดการใช้หน่วยความจำ
- เพิ่มโอกาสในการค้นพบคำตอบที่ดีที่สุด

ดังนั้น ในหลายกรณี ความสำเร็จของการแก้ปัญหาไม่ได้ขึ้นอยู่กับขั้นตอนวิธีเพียงอย่างเดียว แต่ขึ้นอยู่กับวิธีการแทนปัญหานั้นด้วย

2.7 ปัญหาตัวอย่างในงานปัญญาประดิษฐ์

ในช่วงเริ่มต้นของการศึกษาปัญญาประดิษฐ์ นักวิจัยมักเลือกใช้ปัญหาที่มีขนาดเล็ก มีโครงสร้างชัดเจน และสามารถควบคุมเงื่อนไขต่าง ๆ ได้ง่าย ปัญหาเหล่านี้เรียกว่า ปัญหาตัวอย่าง (Toy Problems)

แม้ว่าปัญหาประเภทนี้อาจไม่ได้เกิดขึ้นจริงในชีวิตประจำวันโดยตรง แต่มีบทบาทสำคัญในการศึกษาและพัฒนาขั้นตอนวิธีการค้นหา (Search Algorithms) เนื่องจากช่วยให้นักวิจัยสามารถวิเคราะห์ประสิทธิภาพของขั้นตอนวิธีได้อย่างชัดเจน

เหตุผลที่นักวิจัยในยุคแรกนิยมศึกษาปัญหาตัวอย่าง มีดังนี้

1. ปัญหาที่มีขนาดเล็กและสามารถควบคุมได้ง่าย
2. ช่วยสร้างความเข้าใจในหลักการพื้นฐานของการค้นหา
3. สามารถเปรียบเทียบประสิทธิภาพของขั้นตอนวิธีต่าง ๆ ได้

4. ใช้ทรัพยากรในการคำนวณไม่มาก
 5. สามารถนำแนวคิดที่ได้ไปประยุกต์ใช้กับปัญหาในโลกจริงที่มีความซับซ้อนมากขึ้น
- ปัญหาดังกล่าวที่ได้รับความนิยมมากที่สุด ได้แก่
- 1) ปัญหา 8 ตัวเลข (8-Puzzle Problem)
 - 2) ปัญหา 8 ราชนี (8-Queens Problem)
 - 3) ปัญหาการเดินทางของเซลแมน (Travelling Salesman Problem)

2.7.1 ปัญหา 8 ตัวเลข (8-Puzzle)

ปัญหา 8 ตัวเลข หรือ 8-Puzzle Problem เป็นปัญหาที่ประกอบด้วยแผ่นกระเบื้องตัวเลขจำนวน 8 แผ่น และช่องว่างอีกหนึ่งช่อง บรรจุอยู่ในตารางขนาด 3×3

ในตอนเริ่มต้น ตัวเลขทั้ง 8 ตัวจะถูกจัดเรียงแบบสุ่ม และผู้เล่นจะต้องเลื่อนตัวเลขผ่านช่องว่างเพื่อให้ได้รูปแบบที่ต้องการ

ตัวอย่างสถานะเริ่มต้น

7 2 4

5 □ 6

8 3 1

ตัวอย่างสถานะเป้าหมาย

1 2 3

4 5 6

7 8 □

วัตถุประสงค์ของปัญหาคือ การค้นหาลำดับการเคลื่อนย้ายที่ทำให้สามารถเปลี่ยนจากสถานะเริ่มต้นไปยังสถานะเป้าหมาย โดยใช้จำนวนครั้งในการเคลื่อนที่น้อยที่สุด

1) การแทนปัญหาให้อยู่ในรูปแบบมาตรฐาน

สถานะเริ่มต้น (Initial State)

สถานะเริ่มต้น คือ การจัดเรียงตัวเลขเริ่มต้นบนกระดานขนาด 3×3

ตัวอย่างเช่น

7 2 4

5 □ 6

8 3 1

การกระทำ (Actions)

การกระทำที่เป็นไปได้ คือ

- เลื่อนช่องว่างขึ้น
- เลื่อนช่องว่างลง
- เลื่อนช่องว่างไปทางซ้าย
- เลื่อนช่องว่างไปทางขวา

ทั้งนี้ขึ้นอยู่กับตำแหน่งของช่องว่างในแต่ละสถานะ

แบบจำลองการเปลี่ยนสถานะ (Transition Model)

เมื่อมีการเลือกทิศทางในการเคลื่อนที่ ช่องว่างจะสลับตำแหน่งกับตัวเลขที่อยู่ติดกัน ทำให้เกิดสถานะใหม่

สถานะเป้าหมาย (Goal State)

เป้าหมายคือการจัดเรียงตัวเลขให้อยู่ในลำดับที่กำหนด

1 2 3

4 5 6

7 8 □

ค่าใช้จ่ายของเส้นทาง (Path Cost)

การเคลื่อนย้ายแต่ละครั้งมีค่าใช้จ่ายเท่ากับ 1 หน่วย

ดังนั้น ค่าใช้จ่ายรวมของเส้นทางจะเท่ากับจำนวนครั้งที่ใช้ในการเคลื่อนย้ายทั้งหมด

ความซับซ้อนของปัญหา 8 ตัวเลข

จำนวนรูปแบบการจัดเรียงทั้งหมดของกระดานขนาด 3×3 มีค่าเท่ากับ $9! = 362,880$ รูปแบบ

อย่างไรก็ตาม มีเพียงครึ่งหนึ่งของจำนวนดังกล่าวที่สามารถนำไปสู่คำตอบได้จริง ดังนั้น จำนวนสถานะที่เป็นไปได้จริงมีค่าเท่ากับ $9!/2 = 181,440$ สถานะ

จำนวนสถานะที่มีอยู่เป็นจำนวนมาก ทำให้การค้นหาแบบสุ่ม (Random Search) ไม่มีประสิทธิภาพ และเป็นเหตุผลสำคัญที่ทำให้มีการพัฒนาขั้นตอนวิธีการค้นหาต่าง ๆ เช่น Breadth-First Search และ A* Search

2.7.2 ปัญหา 8 ราชีนี (8-Queens Problem)

ปัญหา 8 ราชีนี เป็นหนึ่งในปัญหาคลาสสิกที่ได้รับความนิยมมากที่สุดในงานด้านปัญญาประดิษฐ์ โจทย์กำหนดให้วางหมากราชีนีจำนวน 8 ตัวลงบนกระดานหมากรุกขนาด 8×8 โดยมีเงื่อนไขว่า ไม่มีราชีนีสองตัวใดสามารถโจมตีกันได้ เนื่องจากหมากราชีนีสามารถเดินได้ทั้ง

- แนวนอน
- แนวตั้ง
- แนวทแยง

ดังนั้น ราชินีทุกตัวจะต้องอยู่คนละแถว คนละคอลัมน์ และคนละแนวทแยง

การแทนปัญหาให้อยู่ในรูปแบบมาตรฐาน

สถานะเริ่มต้น (Initial State)

กระดานหมากรุกขนาด 8×8 ที่ยังไม่มีราชินีถูกวาง

การกระทำ (Actions)

วางราชินีลงบนกระดานครั้งละหนึ่งตัว โดยไม่ให้ขัดแย้งกับตำแหน่งของราชินีที่ถูกวางไว้ก่อนหน้านี้

แบบจำลองการเปลี่ยนสถานะ (Transition Model)

เมื่อมีการวางราชินีใหม่ จะเกิดสถานะใหม่ของกระดานหมากรุก

สถานะเป้าหมาย (Goal State)

วางราชินีครบทั้ง 8 ตัว และไม่มีราชินีคู่ใดโจมตีกันได้

ค่าใช้จ่ายของเส้นทาง (Path Cost)

การวางราชินีแต่ละครั้งมีค่าใช้จ่ายเท่ากับ 1 หน่วย

ความซับซ้อนของปัญหา

หากพิจารณาการวางราชินีทั้ง 8 ตัวบนกระดานขนาด 8×8 จะมีความเป็นไปได้จำนวนมหาศาล

จำนวนการจัดวางทั้งหมดเท่ากับ

$$64! \approx 1.27 \times 10^{89}$$

อย่างไรก็ตาม เมื่อพิจารณาเฉพาะคำตอบที่ถูกต้อง จะพบว่ามีคำตอบทั้งหมด 92 รูปแบบ

ปัญหา 8 ราชินีเป็นตัวอย่างที่ดีของปัญหาที่มีปริภูมิสถานะ (State Space) ขนาดใหญ่ และมักถูกนำมาใช้ในการศึกษาขั้นตอนวิธีแบบย้อนรอย (Backtracking) ขั้นตอนวิธีแบบฮิวริสติก และขั้นตอนวิธีเชิงวิวัฒนาการ

ความสำคัญของปัญหาดังกล่าว

ปัญหา 8 ตัวเลขและปัญหา 8 ราชินี เป็นตัวอย่างของปัญหาที่มีปริภูมิสถานะขนาดใหญ่ (Large State Space) ซึ่งไม่สามารถแก้ปัญหาได้อย่างมีประสิทธิภาพด้วยการสุ่มคำตอบทั้งหมด

ดังนั้น จึงมีการพัฒนาขั้นตอนวิธีการค้นหาหลากหลายรูปแบบเพื่อช่วยลดจำนวนสถานะที่ต้องพิจารณา และเพิ่มประสิทธิภาพในการค้นหาคำตอบ

แนวคิดเหล่านี้ถือเป็นพื้นฐานสำคัญของปัญญาประดิษฐ์แบบดั้งเดิม และถูกนำไปประยุกต์ใช้ในงานด้านต่าง ๆ เช่น การวางแผน การควบคุมหุ่นยนต์ การนำทาง และการเรียนรู้ของเครื่อง

2.8 ปัญหาแบบกำหนดผลลัพธ์ได้แน่นอนและไม่แน่นอน

ปัญหาที่พบในงานด้านปัญญาประดิษฐ์มีความหลากหลายและมีระดับความซับซ้อนแตกต่างกัน ปัจจัยสำคัญประการหนึ่งที่ใช้จำแนกประเภทของปัญหา คือ ความสามารถในการคาดการณ์ผลลัพธ์ของการกระทำ

หากการกระทำเดียวกันให้ผลลัพธ์เหมือนเดิมทุกครั้ง ปัญหานี้จะถูกเรียกว่า ปัญหาแบบกำหนดผลลัพธ์ได้แน่นอน (Deterministic Problem) แต่ถ้าการกระทำเดียวกันอาจให้ผลลัพธ์ที่ต่างกันได้ ปัญหานี้จะเรียกว่า ปัญหาแบบไม่กำหนดผลลัพธ์ได้แน่นอน (Non-Deterministic Problem)

การจำแนกประเภทของปัญหามีความสำคัญ เนื่องจากส่งผลต่อการเลือกขั้นตอนวิธีที่เหมาะสมในการแก้ปัญหา

2.8.1 ปัญหาแบบกำหนดผลลัพธ์ได้แน่นอน (Deterministic Problem)

ปัญหาแบบกำหนดผลลัพธ์ได้แน่นอน หมายถึง ปัญหาที่เมื่อกำหนดสถานะเริ่มต้นและการกระทำเดียวกัน จะได้ผลลัพธ์เหมือนเดิมทุกครั้ง

สามารถอธิบายได้ด้วยสมการ

$$\text{Result}(s,a)=s'$$

โดยที่

s คือ สถานะปัจจุบัน (Current State)

a คือ การกระทำ (Action)

s' คือ สถานะถัดไป (Successor State)

ในปัญหาประเภทนี้ เมื่อกำหนดค่า s และ a แล้ว จะมีสถานะผลลัพธ์เพียงค่าเดียวเสมอ

ตัวอย่างของปัญหาแบบกำหนดผลลัพธ์ได้แน่นอน ได้แก่

- ปัญหา 8 ตัวเลข (8-Puzzle)
- ปัญหา 8 ราชีนี (8-Queens Problem)
- เกมหมากรุก
- การแก้สมการทางคณิตศาสตร์
- การหาเส้นทางบนแผนที่ที่ไม่มีการเปลี่ยนแปลงของสภาพจราจร

ตัวอย่างเช่น

ถ้าหุ่นยนต์อยู่ที่ตำแหน่ง (2,3) และเลือกคำสั่ง

Move Right

ทุกครั้งหุ่นยนต์จะย้ายไปยังตำแหน่ง (3,3)

เสมอ

ดังนั้น ผลลัพธ์สามารถคาดการณ์ได้อย่างสมบูรณ์

ปัญหาประเภทนี้เป็นปัญหาที่นิยมใช้ในการศึกษาขั้นตอนวิธีการค้นหาแบบต่าง ๆ เช่น

- Breadth-First Search
- Depth-First Search

- Uniform-Cost Search
- Greedy Best-First Search
- A* Search

2.8.2 ปัญหาแบบไม่กำหนดผลลัพธ์ได้แน่นอน (Non-Deterministic Problem)

ปัญหาแบบไม่กำหนดผลลัพธ์ได้แน่นอน หมายถึง ปัญหาที่การกระทำเดียวกันอาจนำไปสู่ผลลัพธ์ได้มากกว่าหนึ่งแบบ

กล่าวคือ

$\text{Result}(s,a) \rightarrow \{s_1, s_2, s_3, \dots\}$

ผลลัพธ์ของการกระทำไม่สามารถคาดการณ์ได้อย่างสมบูรณ์

ตัวอย่างเช่น

หุ่นยนต์ที่เคลื่อนที่บนพื้นลื่น

แม้ว่าจะสั่งให้เคลื่อนที่ไปทางขวา แต่ผลลัพธ์อาจเกิดขึ้นได้หลายกรณี เช่น

- เคลื่อนไปทางขวาตามปกติ
- ลื่นไถลไปข้างหน้า
- ลื่นไถลไปทางซ้าย

ดังนั้น การกระทำเดียวกันอาจก่อให้เกิดหลายสถานะถัดไป

ตัวอย่างของปัญหาแบบไม่กำหนดผลลัพธ์ได้แน่นอน ได้แก่

- การควบคุมหุ่นยนต์ในสภาพแวดล้อมจริง
- การพยากรณ์อากาศ
- การลงทุนในตลาดหุ้น
- การวางแผนทางธุรกิจ
- การขับขี่ยานต์อัตโนมัติ

ปัญหาประเภทนี้มีความซับซ้อนสูงกว่าปัญหาแบบกำหนดผลลัพธ์ได้แน่นอน เนื่องจากต้องพิจารณาความเป็นไปได้หลายรูปแบบพร้อมกัน

2.8.3 ปัญหาแบบสุ่ม (Stochastic Problem)

ในหลายกรณี ผลลัพธ์ของการกระทำไม่ได้เกิดขึ้นแบบสุ่มอย่างสมบูรณ์ แต่สามารถอธิบายด้วยความน่าจะเป็นได้ ปัญหาประเภทนี้เรียกว่า ปัญหาแบบสุ่ม (Stochastic Problem)

ตัวอย่างเช่น การพยากรณ์สภาพอากาศ อาจคาดการณ์ได้ว่า

- โอกาสเกิดฝนตก 80%
- โอกาสที่ท้องฟ้าแจ่มใส 20%

หรือในการโยนลูกเต๋า

- โอกาสที่หน้าลูกเต๋าคจะออกเลข 1 เท่ากับ $1/6$
- โอกาสที่หน้าลูกเต๋าคจะออกเลข 2 เท่ากับ $1/6$

แม้ว่าจะไม่สามารถระบุผลลัพธ์ได้อย่างแน่นอน แต่สามารถอธิบายด้วยความน่าจะเป็นของเหตุการณ์ต่าง ๆ ได้ ปัญหาแบบสุ่มพบได้มากในโลกแห่งความเป็นจริง เช่น

- การลงทุนทางการเงิน
- การวิเคราะห์ความเสี่ยง
- การพยากรณ์โรค
- ระบบแนะนำสินค้า
- รถยนต์ขับเคลื่อนอัตโนมัติ

2.8.4 การเปรียบเทียบปัญหาทั้งสามประเภท

คุณลักษณะต่าง ๆ ของปัญหาทั้ง 3 ประเภทถูกแสดงการเปรียบเทียบไว้ดังตารางที่ 2.6 นอกจากนี้วิธีการแก้ปัญหาต่าง ๆ ยังสามารถแสดงไว้ดังรูปที่ 2.3

ตารางที่ 2.6 เปรียบเทียบลักษณะของปัญหาแบบกำหนดผลลัพธ์ได้แน่นอน ปัญหาแบบไม่กำหนดผลลัพธ์ได้ และปัญหาแบบสุ่ม

คุณลักษณะ	Deterministic	Non-Deterministic	Stochastic
ผลลัพธ์ของการกระทำ	มีเพียงหนึ่งผลลัพธ์	หลายผลลัพธ์	หลายผลลัพธ์พร้อมความน่าจะเป็น
ความสามารถในการคาดการณ์	สูง	ต่ำ	ปานกลาง
การใช้ความน่าจะเป็น	ไม่ใช่	ไม่จำเป็น	ใช่
ความซับซ้อนของปัญหา	ต่ำ	สูง	สูง
ตัวอย่าง	8-Puzzle, 8-Queens	หุ่นยนต์บนพื้นที่ลื่น	พยากรณ์อากาศ

2.8.5 ความสำคัญของการจำแนกประเภทของปัญหา

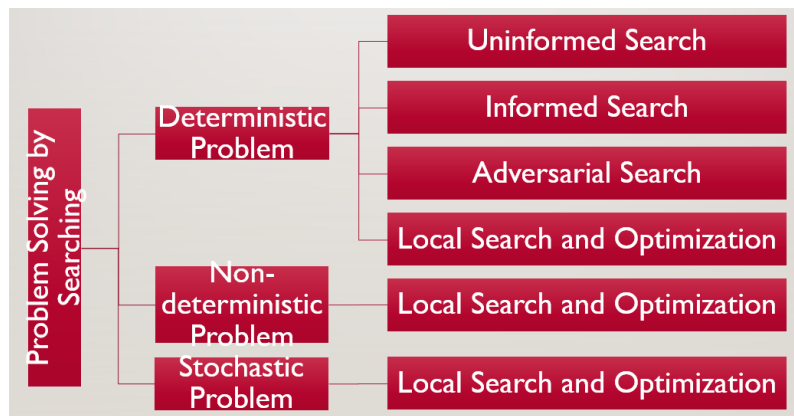
การจำแนกประเภทของปัญหามีความสำคัญ เนื่องจากขั้นตอนวิธีที่เหมาะสมสำหรับแต่ละประเภทของปัญหามีความแตกต่างกัน ตัวอย่างเช่น ปัญหาแบบกำหนดผลลัพธ์ได้แน่นอน มักใช้

- Breadth-First Search
- Depth-First Search

- Greedy Search
- A* Search
- ปัญหาแบบไม่กำหนดผลลัพธ์ได้และปัญหาแบบสุ่ม มักใช้
- Markov Decision Process (MDP)
- Reinforcement Learning
- Dynamic Programming
- Probabilistic Reasoning

ดังนั้น การทำความเข้าใจลักษณะของปัญหาจึงเป็นขั้นตอนสำคัญก่อนการเลือกวิธีการแก้ปัญหาที่

เหมาะสม



รูปที่ 2.3 เปรียบเทียบปัญหาทั้ง 3 ประเภท และวิธีการค้นหาคำตอบเพื่อแก้ปัญหาทั้ง 3 ประเภท

2.9 การวัดประสิทธิภาพของการค้นหา

ในการแก้ปัญหาด้วยปัญญาประดิษฐ์ อาจมีขั้นตอนวิธีหลายชนิดที่สามารถค้นหาคำตอบได้ แต่ขั้นตอนวิธีแต่ละชนิดมีประสิทธิภาพแตกต่างกัน ทั้งในด้านเวลา หน่วยความจำ และคุณภาพของคำตอบที่ได้ ดังนั้นจึงจำเป็นต้องมีเกณฑ์สำหรับใช้เปรียบเทียบประสิทธิภาพของขั้นตอนวิธีการค้นหาต่าง ๆ

โดยทั่วไป การประเมินประสิทธิภาพของขั้นตอนวิธีการค้นหาในงานด้านปัญญาประดิษฐ์นิยมพิจารณาจากคุณสมบัติหลัก 4 ประการ ได้แก่

1. ความสามารถการันตีหาคำตอบได้ (Completeness)
2. ความสามารถในการรับประกันคำตอบที่ดีที่สุด (Optimality)
3. ความซับซ้อนทางเวลา (Time Complexity)
4. ความซับซ้อนทางหน่วยความจำ (Space Complexity)

คุณสมบัติทั้งสี่ประการนี้เป็นเกณฑ์มาตรฐานที่ใช้ในการวิเคราะห์และเปรียบเทียบขั้นตอนวิธีการค้นหาต่าง ๆ

2.9.1 ความสามารถค้นหาคำตอบได้ (Completeness)

ความสามารถของการค้นหา หมายถึง ความสามารถของขั้นตอนวิธีในการรับประกันว่าจะสามารถค้นพบคำตอบได้ หากคำตอบนั้นมีอยู่จริง กล่าวคือ หากปัญหามีคำตอบ ขั้นตอนวิธีที่มีคุณสมบัติ Completeness จะต้องสามารถค้นพบคำตอบได้ในที่สุด ตัวอย่างเช่น การค้นหาแบบ Breadth-First Search (BFS) เป็นขั้นตอนวิธีที่มีความสมบูรณ์ เนื่องจากจะค้นหาสถานะทุกสถานะในระดับเดียวกันก่อนเสมอ ดังนั้น หากมีคำตอบอยู่ในปริภูมิสถานะ ขั้นตอนวิธีนี้จะสามารถค้นพบคำตอบได้

ในทางตรงกันข้าม Depth-First Search (DFS) อาจไม่สามารถค้นพบคำตอบได้ หากเกิดการวนซ้ำไม่สิ้นสุด หรือมีความลึกของต้นไม้การค้นหามากเกินไป ดังนั้น ความสมบูรณ์จึงเป็นคุณสมบัติพื้นฐานที่สำคัญของขั้นตอนวิธีการค้นหา

2.9.2 ความสามารถในการรับประกันคำตอบที่ดีที่สุด (Optimality)

Optimality หมายถึง ความสามารถของขั้นตอนวิธีในการรับประกันว่าคำตอบที่ค้นพบเป็นคำตอบที่ดีที่สุด โดยทั่วไป คำตอบที่ดีที่สุดหมายถึงคำตอบที่มีค่าใช้จ่ายรวมต่ำที่สุด หรือใช้จำนวนขั้นตอนน้อยที่สุด

ตัวอย่างเช่น สมมติว่ามีเส้นทางสองเส้นทางจากจุดเริ่มต้นไปยังจุดหมาย

เส้นทางที่ 1

$A \rightarrow B \rightarrow G$

ค่าใช้จ่ายรวม = 15

เส้นทางที่ 2

$A \rightarrow C \rightarrow D \rightarrow G$

ค่าใช้จ่ายรวม = 10

แม้ว่าทั้งสองเส้นทางจะสามารถไปถึงเป้าหมายได้ แต่เส้นทางที่สองถือเป็นคำตอบที่ดีกว่า

ขั้นตอนวิธีที่มีคุณสมบัติ Optimality จะต้องสามารถค้นพบเส้นทางที่สองได้

ตัวอย่างของขั้นตอนวิธีที่สามารถรับประกันคำตอบที่ดีที่สุด ได้แก่

- Breadth-First Search (เมื่อค่าใช้จ่ายแต่ละขั้นเท่ากัน)
- Uniform-Cost Search
- A* Search (เมื่อใช้ฮิวริสติกที่เหมาะสม)

ส่วน Greedy Best-First Search ไม่สามารถรับประกันได้ว่าคำตอบที่ได้จะดีที่สุดเสมอไป

2.9.3 ความซับซ้อนทางเวลา (Time Complexity)

ความซับซ้อนทางเวลา (Time Complexity) หมายถึง จำนวนการคำนวณหรือจำนวนสถานะที่ขั้นตอนวิธีต้องพิจารณาในการค้นหาคำตอบ

ในทางปฏิบัติ ไม่นิยมวัดเวลาเป็นวินาที เนื่องจากความเร็วในการประมวลผลขึ้นอยู่กับฮาร์ดแวร์และสภาพแวดล้อมของระบบ ดังนั้นจึงนิยมใช้สัญกรณ์โอใหญ่ (Big-O Notation) ในการอธิบายความซับซ้อนทางเวลา

ตัวอย่างเช่น

- $O(1)$ เวลาคงที่ (Constant Time)
- $O(\log n)$ เวลาลอการิทึม (Logarithmic Time)
- $O(n)$ เวลาเชิงเส้น (Linear Time)
- $O(n^2)$ เวลากำลังสอง (Quadratic Time)
- $O(n^3)$ เวลาลูกบาศก์ (Cubic Time)
- $O(2^n)$ เวลาเลขชี้กำลัง (Exponential Time)
- $O(n!)$ เวลาแฟกทอเรียล (Factorial Time)

โดยทั่วไป ขั้นตอนวิธีที่มีความซับซ้อนทางเวลาดำกว่าจะมีประสิทธิภาพสูงกว่า ตัวอย่างเช่น

- หากมีข้อมูลจำนวน 1,000 รายการ ขั้นตอนวิธีที่มี Time Complexity เท่ากับ $O(n)$ จะใช้จำนวนการคำนวณประมาณ 1,000 ครั้ง
- แต่ถ้ามี Time Complexity เท่ากับ $O(n^2)$ จำนวนการคำนวณจะเพิ่มขึ้นเป็นประมาณ 1,000,000 ครั้ง

ดังนั้น ความซับซ้อนทางเวลาจึงมีผลโดยตรงต่อประสิทธิภาพของขั้นตอนวิธี

2.9.4 ความซับซ้อนทางหน่วยความจำ (Space Complexity)

ความซับซ้อนทางหน่วยความจำ (Space Complexity) หมายถึง ปริมาณหน่วยความจำที่ต้องใช้ระหว่างกระบวนการค้นหา ตัวอย่างเช่น Breadth-First Search จำเป็นต้องเก็บโหนดทั้งหมดในแต่ละระดับไว้ในหน่วยความจำ ทำให้ใช้หน่วยความจำจำนวนมาก

ในขณะที่ Depth-First Search เก็บเฉพาะเส้นทางที่กำลังสำรวจอยู่ จึงใช้หน่วยความจำน้อยกว่า แม้ว่าขั้นตอนวิธีสองชนิดอาจใช้เวลาค้นหาใกล้เคียงกัน แต่ขั้นตอนวิธีที่ใช้หน่วยความจำน้อยกว่าจะเหมาะสมกับระบบที่มีทรัพยากรจำกัด

ดังนั้น การเลือกขั้นตอนวิธีที่เหมาะสมจึงต้องพิจารณาทั้งเวลาและหน่วยความจำร่วมกัน

2.9.5 การเปรียบเทียบคุณสมบัติของขั้นตอนวิธีการค้นหา

ตารางที่ 2.7 คุณสมบัติที่ใช้วัดประสิทธิภาพของขั้นตอนวิธีการค้นหา

คุณสมบัติ	ความหมาย
Completeness	สามารถรับประกันว่าจะพบคำตอบ หากคำตอบมีอยู่จริง

คุณสมบัติ	ความหมาย
Optimality	สามารถรับประกันว่าคำตอบที่ได้ดีที่สุด
Time Complexity	จำนวนการคำนวณหรือเวลาที่ใช้ในการค้นหา
Space Complexity	ปริมาณหน่วยความจำที่ใช้ระหว่างการค้นหา

คุณสมบัติทั้งสี่ประการนี้ถือเป็นเกณฑ์มาตรฐานที่ใช้ในการเปรียบเทียบขั้นตอนวิธีการค้นหาต่าง ๆ ซึ่งจะถูกนำมาใช้ในการวิเคราะห์ Breadth-First Search, Depth-First Search, Greedy Best-First Search และ A* Search ในหัวข้อถัดไป

2.9.6 ปัจจัยที่มีผลต่อประสิทธิภาพของการค้นหา

ประสิทธิภาพของขั้นตอนวิธีไม่ได้ขึ้นอยู่กับตัวขั้นตอนวิธีเพียงอย่างเดียว แต่ยังขึ้นอยู่กับลักษณะของปัญหาอีกด้วย ปัจจัยสำคัญที่มีผลต่อประสิทธิภาพ ได้แก่

จำนวนกิ่งของต้นไม้การค้นหา (Branching Factor)

Branching Factor เขียนแทนด้วย b หมายถึง จำนวนโหนดลูกโดยเฉลี่ยของแต่ละโหนด ยิ่งค่า b มีค่ามาก จำนวนสถานะที่ต้องสำรวจก็จะเพิ่มขึ้นอย่างรวดเร็ว

ความลึกของคำตอบ (Depth)

เขียนแทนด้วย d
หมายถึง จำนวนระดับจากโหนดเริ่มต้นไปจนถึงโหนดเป้าหมาย
หากคำตอบอยู่ลึกมาก ขั้นตอนวิธีอาจต้องใช้เวลาในการค้นหา

ขนาดของปริภูมิสถานะ (State Space)

ปัญหาที่มีจำนวนสถานะมาก เช่น ปัญหา 8 ราชินี หรือเกมหมากรุก จะมีความซับซ้อนสูงกว่าปัญหาขนาดเล็ก

คุณภาพของฟังก์ชันฮิวริสติก

สำหรับการค้นหาแบบใช้ข้อมูลนำทาง ประสิทธิภาพของขั้นตอนวิธีขึ้นอยู่กับคุณภาพของฟังก์ชันฮิวริสติก หากสามารถประมาณระยะทางไปยังเป้าหมายได้ดี จำนวนโหนดที่ต้องสำรวจก็จะลดลง

ดังนั้น การวิเคราะห์ประสิทธิภาพของขั้นตอนวิธีการค้นหาจึงจำเป็นต้องพิจารณาปัจจัยหลายด้านร่วมกัน ไม่ใช่เพียงความเร็วในการคำนวณเท่านั้น

2.10 ความซับซ้อนของปัญหา

ในการแก้ปัญหาด้วยคอมพิวเตอร์ นอกจากการพิจารณาว่าสามารถหาคำตอบได้หรือไม่แล้ว ยังต้องพิจารณาด้วยว่าการหาคำตอบนั้นต้องใช้เวลาและทรัพยากรมากเพียงใด เนื่องจากปัญหาบางประเภทแม้จะมีคำตอบอยู่จริง แต่การค้นหาคำตอบอาจใช้เวลานานมากจนไม่สามารถนำไปใช้งานได้จริง

ศาสตร์ที่ใช้ศึกษาและวิเคราะห์ความยากง่ายของปัญหาและประสิทธิภาพของขั้นตอนวิธี เรียกว่า ทฤษฎีความซับซ้อน (Complexity Theory)

ทฤษฎีความซับซ้อนมีบทบาทสำคัญในการออกแบบขั้นตอนวิธี การวิเคราะห์ประสิทธิภาพของ โปรแกรม และการเลือกวิธีการแก้ปัญหาที่เหมาะสม

2.10.1 ความซับซ้อนทางเวลา (Time Complexity)

ความซับซ้อนทางเวลา (Time Complexity) หมายถึง จำนวนการดำเนินการหรือจำนวนครั้งของการ คำนวณที่เพิ่มขึ้นตามขนาดของข้อมูลนำเข้า

ในทางปฏิบัติไม่นิยามวัดเวลาเป็นวินาที เนื่องจากเวลาในการทำงานขึ้นอยู่กับ

- ความเร็วของหน่วยประมวลผล
- ระบบปฏิบัติการ
- ภาษาโปรแกรม
- ฮาร์ดแวร์ที่ใช้

ดังนั้นจึงนิยมใช้ สัญกรณ์โอใหญ่ (Big-O Notation) เพื่ออธิบายแนวโน้มการเพิ่มขึ้นของเวลาในการ คำนวณ

2.10.2 สัญกรณ์โอใหญ่ (Big-O Notation)

Big-O Notation เป็นวิธีการอธิบายอัตราการเติบโตของเวลาในการทำงานเมื่อขนาดของข้อมูลเพิ่มขึ้น โดยทั่วไปนิยมใช้สัญลักษณ์ $O(f(n))$ เมื่อ

n คือ ขนาดของข้อมูลนำเข้า

$f(n)$ คือ อัตราการเพิ่มขึ้นของจำนวนการคำนวณ

ตัวอย่างเช่น $O(n)$ หมายความว่า จำนวนการคำนวณเพิ่มขึ้นเป็นสัดส่วนโดยตรงกับจำนวนข้อมูล

เวลาคงที่ (Constant Time)

$O(1)$ จำนวนการคำนวณไม่ขึ้นอยู่กับขนาดของข้อมูล ตัวอย่าง การเข้าถึงสมาชิกในอาร์เรย์ $A[5]$ ไม่ว่า ข้อมูลจะมีขนาดเท่าใด เวลาที่ใช้จะใกล้เคียงเดิมเสมอ

เวลาเชิงเส้น (Linear Time)

$O(n)$ จำนวนการคำนวณเพิ่มขึ้นตามจำนวนข้อมูล ตัวอย่าง การค้นหาข้อมูลแบบเรียงลำดับทีละตัว Linear Search

- เวลากำลังสอง (Quadratic Time)

$O(n^2)$ จำนวนการคำนวณเพิ่มขึ้นตามกำลังสองของจำนวนข้อมูล ตัวอย่าง Bubble Sort

- เวลาพหุนาม (Polynomial Time)

$O(n^k)$ เมื่อ k เป็นจำนวนเต็มบวก ขั้นตอนวิธีประเภทนี้ยังถือว่าเป็นปัญหาที่สามารถแก้ไขได้อย่างมีประสิทธิภาพ ตัวอย่าง

- Merge Sort
- Dijkstra Algorithm
- Matrix Multiplication

- เวลาเลขชี้กำลัง (Exponential Time)

$O(2^n)$ เวลาในการคำนวณเพิ่มขึ้นอย่างรวดเร็วเมื่อขนาดของข้อมูลเพิ่มขึ้น ตัวอย่าง การค้นหาทุกความเป็นไปได้ (Brute Force)

- เวลาแฟกทอเรียล (Factorial Time)

$O(n!)$ เป็นความซับซ้อนที่เพิ่มขึ้นอย่างรวดเร็วมาก ตัวอย่าง ปัญหาการเดินทางของเซลแมนที่ใช้การค้นหาทุกเส้นทาง

2.10.3 การเปรียบเทียบความซับซ้อนทางเวลา

เมื่อจำนวนข้อมูลมีค่าเท่ากับ 100 จะได้จำนวนการคำนวณโดยประมาณดังนี้

Time Complexity	จำนวนการคำนวณโดยประมาณ
$O(1)$	1
$O(\log n)$	7
$O(n)$	100
$O(n \log n)$	664
$O(n^2)$	10,000
$O(n^3)$	1,000,000
$O(2^n)$	1.27×10^{30}
$O(n!)$	9.33×10^{157}

จะเห็นว่าเมื่อขนาดของข้อมูลเพิ่มขึ้น ความแตกต่างของประสิทธิภาพระหว่างขั้นตอนวิธีจะเพิ่มขึ้นอย่างมาก ดังนั้น การเลือกขั้นตอนวิธีที่มีความซับซ้อนต่ำจึงมีความสำคัญอย่างยิ่ง

- 2.10.4 กลุ่มปัญหา P

P (Polynomial Time) คือ กลุ่มของปัญหาที่สามารถหาคำตอบได้ภายในเวลาแบบพหุนาม กล่าวคือ มีขั้นตอนวิธีที่สามารถแก้ปัญหได้ภายในเวลา $O(n)$ $O(n^2)$ $O(n^3)$ หรือ $O(n^k)$ ตัวอย่างปัญหาในกลุ่ม P ได้แก่

- การเรียงลำดับข้อมูล
- การค้นหาข้อมูล

- การหาเส้นทางสั้นที่สุดด้วย Dijkstra Algorithm
- การหาค่าเฉลี่ยของข้อมูล

ปัญหาในกลุ่ม P ถือว่าเป็นปัญหาที่สามารถแก้ไขได้อย่างมีประสิทธิภาพ

• 2.10.5 กลุ่มปัญหา NP

NP (Nondeterministic Polynomial Time) คือ กลุ่มของปัญหาที่อาจหาคำตอบได้ยาก แต่เมื่อได้รับคำตอบแล้ว สามารถตรวจสอบความถูกต้องได้ภายในเวลาแบบพหุนาม กล่าวคือ การตรวจสอบคำตอบทำได้ง่ายกว่าการหาคำตอบ หรืออธิบายอีกแบบได้ว่า ถ้ามีคนส่งคำตอบของปัญหาประเภท NP มาให้ จะสามารถ "ตรวจสอบว่าถูกหรือไม่" ได้ในเวลาแบบพหุนาม แต่ปัญหา NP วิธีการได้มาซึ่งคำตอบยาก และอาจไม่สามารถแก้ได้ในเวลาพหุนาม ตัวอย่างปัญหาในกลุ่ม NP ได้แก่

- Sudoku
- Subset Sum Problem
- Traveling Salesman Problem
- Knapsack Problem

ในปัจจุบัน ยังไม่มีการพิสูจน์ว่า $P = NP$ หรือ $P \neq NP$ ซึ่งถือเป็นหนึ่งในปัญหาทางคณิตศาสตร์ที่สำคัญที่สุดของโลก

• 2.10.6 กลุ่มปัญหา NP-Complete

NP-Complete คือ กลุ่มของปัญหาที่ยากที่สุดในกลุ่ม NP หากสามารถค้นพบขั้นตอนวิธีที่แก้ปัญหา NP-Complete ได้ในเวลาแบบพหุนาม จะสามารถนำวิธีการนั้นไปแก้ปัญหาย่อยทุกปัญหาในกลุ่ม NP ได้ ดังนั้น ถ้า NP-Complete สามารถแก้ได้ในเวลาแบบพหุนาม จะทำให้ $P = NP$ ตัวอย่างปัญหา NP-Complete ได้แก่

- Traveling Salesman Problem
- Knapsack Problem
- Boolean Satisfiability Problem (SAT)
- Vertex Cover Problem
- Hamiltonian Cycle Problem

ปัจจุบันยังไม่มีขั้นตอนวิธีที่สามารถแก้ปัญหา NP-Complete ได้อย่างมีประสิทธิภาพ

• 2.10.7 กลุ่มปัญหา NP-Hard

NP-Hard เป็นกลุ่มของปัญหาที่มีความยากอย่างน้อยเท่ากับปัญหา NP-Complete ปัญหาในกลุ่มนี้อาจไม่จำเป็นต้องสามารถตรวจสอบคำตอบได้ภายในเวลาแบบพหุนาม กล่าวคือ

$$NP\text{-Complete} \subseteq NP\text{-Hard}$$

ตัวอย่างปัญหาในกลุ่ม NP-Hard ได้แก่

- การเล่นหมากรุก

- เกมโกะ
- Traveling Salesman Problem แบบหาค่าที่ดีที่สุด
- ปัญหาการจัดตารางเรียน
- ปัญหาการวางแผนการผลิต

ปัญหาเหล่านี้มักต้องใช้วิธีประมาณค่า (Approximation Algorithm) หรือวิธีการหาคำตอบที่ใกล้เคียงที่สุด (Heuristic Algorithm)

2.10.8 ความสัมพันธ์ระหว่าง P, NP, NP-Complete และ NP-Hard

ความสัมพันธ์ของกลุ่มปัญหาทั้งสี่ดังรูปที่ 2.4 สามารถสรุปได้ดังนี้

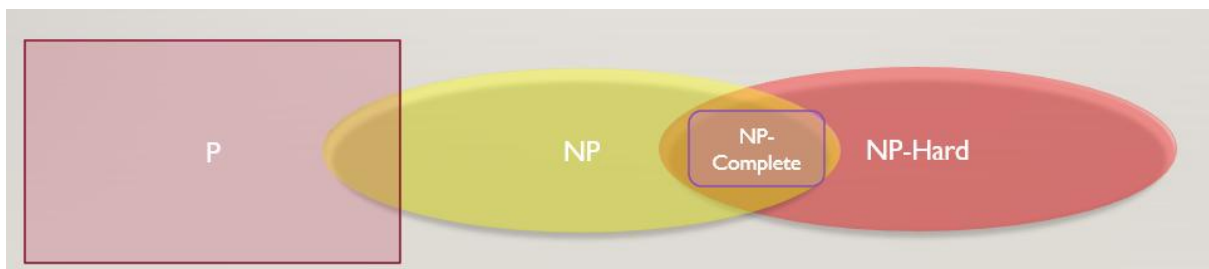
$$P \subseteq NP$$

$$NP\text{-Complete} \subseteq NP$$

$$NP\text{-Complete} \subseteq NP\text{-Hard}$$

กล่าวคือ

- ทุกปัญหาในกลุ่ม P อยู่ในกลุ่ม NP
- NP-Complete เป็นส่วนหนึ่งของ NP
- NP-Hard เป็นกลุ่มปัญหาที่กว้างกว่า NP-Complete



รูปที่ 2.4 ความสัมพันธ์ระหว่าง P, NP, NP-Complete และ NP-Hard

ตารางที่ 2.8 เปรียบเทียบกลุ่มปัญหาต่าง ๆ

กลุ่มปัญหา	ความหมาย	ตัวอย่าง
P	แก้ได้ง่ายและตรวจสอบได้ง่าย	Sorting, Dijkstra
NP	ตรวจสอบคำตอบได้ง่าย	Sudoku, Subset Sum
NP-Complete	ปัญหาที่ยากที่สุดใน NP	TSP, Knapsack
NP-Hard	ยากเท่ากับหรือยากกว่า NP-Complete	Chess, Go

การศึกษาทฤษฎีความซับซ้อนช่วยให้นักวิจัยสามารถวิเคราะห์ความเป็นไปได้ในการแก้ปัญหา และเลือกแนวทางที่เหมาะสมสำหรับปัญหาแต่ละประเภท

2.11 การค้นหาแบบไม่ใช้ข้อมูลนำทาง

การค้นหาแบบไม่ใช้ข้อมูลนำทาง (Uninformed Search) หรือบางครั้งเรียกว่า การค้นหาแบบตาบอด (Blind Search) เป็นวิธีการค้นหาที่อาศัยเพียงข้อมูลพื้นฐานของปัญหา โดยไม่มีข้อมูลเพิ่มเติมหรือความรู้เชิงประสบการณ์เกี่ยวกับตำแหน่งของคำตอบ

กล่าวคือ ขั้นตอนวิธีประเภทนี้ไม่ทราบว่าจะเลือกทิศทางใดเพื่อไปสู่เป้าหมาย จึงจำเป็นต้องสำรวจสถานะต่าง ๆ ตามกฎที่กำหนดไว้ล่วงหน้า

การค้นหาแบบไม่ใช้ข้อมูลนำทางเป็นพื้นฐานสำคัญของการแก้ปัญหาในปัญญาประดิษฐ์ และเป็นจุดเริ่มต้นของการพัฒนาขั้นตอนวิธีการค้นหาแบบใช้ข้อมูลนำทางในภายหลัง

ขั้นตอนวิธีในกลุ่มนี้ที่ได้รับความนิยม ได้แก่

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform-Cost Search (UCS)
- Bidirectional Search

แม้ว่าขั้นตอนวิธีเหล่านี้จะไม่มีข้อมูลช่วยนำทาง แต่สามารถรับประกันการค้นพบคำตอบได้ในหลายกรณี และยังเป็นพื้นฐานสำคัญสำหรับการศึกษาเทคนิคการค้นหาขั้นสูงในบทต่อไป

2.11.1 การค้นหาแบบแนวกว้าง (Breadth-First Search)

Breadth-First Search หรือ BFS เป็นขั้นตอนวิธีที่ทำการค้นหาโหนดทุกโหนดในระดับเดียวกันก่อน แล้วจึงขยายไปยังระดับที่ลึกขึ้น ลักษณะการค้นหาของ BFS มีลักษณะเหมือนการแผ่ขยายออกเป็นวงกว้างจากจุดเริ่มต้น ขั้นตอนวิธีนี้ใช้โครงสร้างข้อมูลแบบคิว (Queue) ซึ่งมีหลักการทำงานแบบเข้าก่อนออกก่อน (First In First Out: FIFO) โดยแสดงตัวอย่างลำดับการค้นหาดังรูปที่ 2.5

หลักการทำงานของ Breadth-First Search

1. นำโหนดเริ่มต้นเข้าสู่คิว
2. นำโหนดตัวแรกออกจากคิว
3. ตรวจสอบว่าเป็นโหนดเป้าหมายหรือไม่
4. ถ้ายังไม่ใช่ ให้เพิ่มโหนดลูกทั้งหมดเข้าสู่คิว
5. ทำซ้ำจนกว่าจะพบคำตอบหรือไม่มีโหนดเหลืออยู่

ตัวอย่างลำดับการเข้าถึงโหนด A

ระดับที่ 1 เป็น B, C

ระดับที่ 2 เป็น D, E, F, G

ดังนั้นลำดับการเข้าถึงจะเป็น

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

ข้อดีของ Breadth-First Search

- มีคุณสมบัติการันตีในการหาคำตอบได้ (Completeness)
- รับประกันคำตอบที่ดีที่สุดในเมื่อค่าใช้จ่ายแต่ละขั้นเท่ากัน
- เหมาะกับปัญหาที่คำตอบอยู่ไม่ลึกมาก

ข้อเสียของ Breadth-First Search

- ใช้หน่วยความจำจำนวนมาก
- จำนวนโหนดเพิ่มขึ้นอย่างรวดเร็วเมื่อความลึกเพิ่มขึ้น

ความซับซ้อน

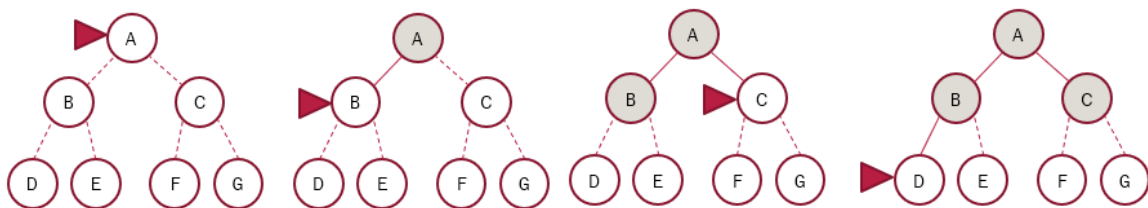
Time Complexity = $O(b^d)$

Space Complexity = $O(b^d)$

เมื่อ

b คือ องค์ประกอบโหนดในแต่ละสาขา (Branching Factor)

d คือ ความลึกของคำตอบ



รูปที่ 2.5 การค้นหาแบบแนวกว้าง

2.11.2 การค้นหาแบบแนวลึก (Depth-First Search)

Depth-First Search หรือ DFS เป็นขั้นตอนวิธีที่สำรวจโหนดในแนวลึกก่อน เมื่อไม่สามารถเดินทางต่อไปได้จึงย้อนกลับมายังโหนดก่อนหน้า DFS ใช้โครงสร้างข้อมูลแบบสแต็ก (Stack) ซึ่งทำงานแบบเข้าที่หลังออกก่อน (Last In First Out: LIFO) โดยแสดงตัวอย่างลำดับการค้นหาดังรูปที่ 2.6 รูปที่ 2.5

หลักการทำงานของ Depth-First Search

1. เริ่มต้นจากโหนดเริ่มต้น
2. เลือกโหนดลูกตัวแรก
3. เดินหน้าลงไปตามแนวลึกให้มากที่สุด
4. เมื่อไม่พบโหนดลูกเพิ่มเติม ให้ย้อนกลับไปยังโหนดก่อนหน้า
5. ทำซ้ำจนกว่าจะพบคำตอบ

ตัวอย่างลำดับการเข้าถึง

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

ข้อดีของ Depth-First Search

- ใช้หน่วยความจำน้อย
- เหมาะกับปัญหาที่คำตอบอยู่ลึก

ข้อเสียของ Depth-First Search

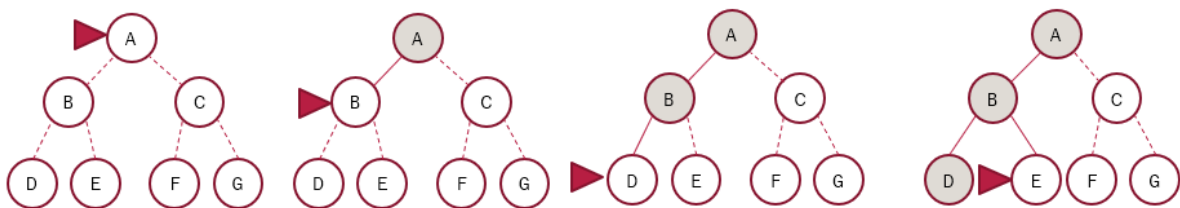
- ไม่รับประกันว่าจะพบคำตอบ
- อาจเกิดการวนซ้ำไม่สิ้นสุด
- ไม่รับประกันคำตอบที่ดีที่สุด

ความซับซ้อน

Time Complexity = $O(b^m)$

Space Complexity = $O(bm)$

เมื่อ m คือ ความลึกสูงสุดของต้นไม้การค้นหา



รูปที่ 2.6 การค้นหาแบบแนวลึก

2.11.3 การค้นหาแบบพิจารณาค่าใช้จ่าย (Uniform-Cost Search)

Uniform-Cost Search หรือ UCS เป็นการค้นหาที่เลือกโหนดที่มีค่าใช้จ่ายสะสมต่ำที่สุดก่อน ขั้นตอนวิธีนี้เป็นการขยายแนวคิดของ Breadth-First Search ให้รองรับกรณีที่ค่าใช้จ่ายของแต่ละเส้นทางไม่เท่ากัน โดยแสดงตัวอย่างลำดับการค้นหาดังรูปที่ 2.7 การค้นหาแบบพิจารณาค่าใช้จ่ายจะมีลักษณะเหมือนกับการค้นหาแบบไดสตาร์ (Dijkstra Algorithm)

หลักการทำงาน

1. เริ่มต้นจากโหนดเริ่มต้น
2. เลือกโหนดที่มีค่าใช้จ่ายสะสมต่ำที่สุด
3. ขยายโหนดนั้น
4. ทำซ้ำจนกว่าจะพบเป้าหมาย

ตัวอย่าง

$A \rightarrow B$ ค่าใช้จ่าย 3

$A \rightarrow C$ ค่าใช้จ่าย 1

$C \rightarrow G$ ค่าใช้จ่าย 4

$B \rightarrow G$ ค่าใช้จ่าย 10

UCS จะเลือกเส้นทาง

$A \rightarrow C \rightarrow G$

เนื่องจากมีค่าใช้จ่ายรวมเท่ากับ 5

ซึ่งต่ำกว่าเส้นทางอื่น

ข้อดี

- มีคุณสมบัติการันตีหาคำตอบได้ (Completeness)
- รับประกันคำตอบที่ดีที่สุด

ข้อเสีย

- ใช้เวลาในการค้นหาเยอะ
- ใช้หน่วยความจำสูง

ความซับซ้อน

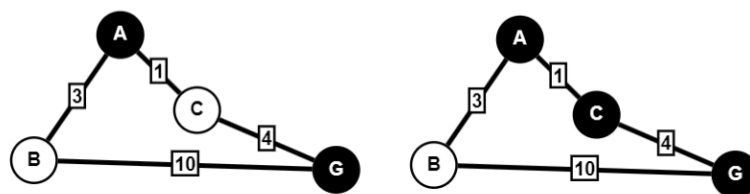
Time Complexity $O(b^{1+[C^*/\epsilon]})$

Space Complexity $O(b^{1+[C^*/\epsilon]})$

เมื่อ

C^* คือค่าใช้จ่ายของคำตอบที่ดีที่สุด

ϵ คือค่าใช้จ่ายต่ำสุดของแต่ละขั้น



รูปที่ 2.7 การค้นหาแบบพิจารณาค่าใช้จ่าย

2.11.4 การค้นหาแบบสองทิศทาง (Bidirectional Search)

Bidirectional Search เป็นขั้นตอนวิธีที่ทำการค้นหาจากทั้งสองด้านพร้อมกัน โดยแสดงโครงสร้างการค้นหาดังรูปที่ 2.8 สามารถอธิบายการทำงานดังนี้

- เริ่มต้นค้นหาจากสถานะเริ่มต้นไปยังเป้าหมาย พร้อมกับค้นหาจากสถานะเป้าหมายย้อนกลับมายังจุดเริ่มต้น โดยการค้นหาจะสิ้นสุดเมื่อทั้งสองส่วนมาพบกัน จะสามารถสร้างคำตอบสมบูรณ์ได้

ข้อดี

- ลดจำนวนโหนดที่ต้องสำรวจ
- ทำงานได้รวดเร็วกว่าการค้นหาแบบทิศทางเดียว

ข้อเสีย

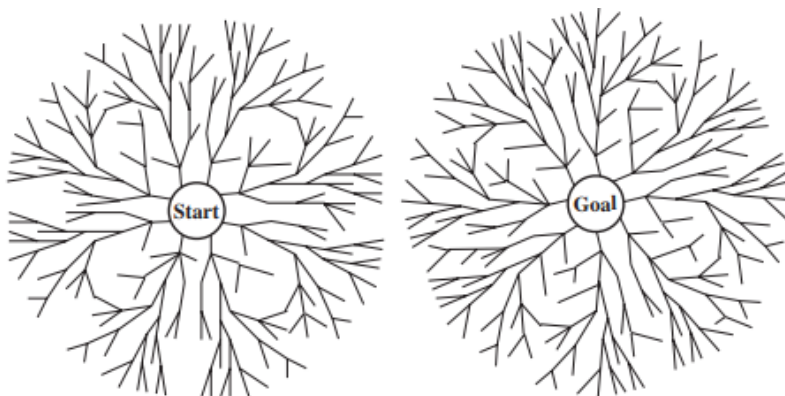
- จำเป็นต้องทราบตำแหน่งเป้าหมายล่วงหน้า
- การค้นหาแบบย้อนกลับอาจทำได้ยากในบางปัญหา

ความซับซ้อน

Time Complexity $O(b^{d/2})$

Space Complexity $O(b^{d/2})$

ซึ่งมีประสิทธิภาพดีกว่า Breadth-First Search ที่มีความซับซ้อนเท่ากับ $O(b^d)$



รูปที่ 2.8 โครงสร้างการค้นหาแบบสองทิศทาง

2.11.5 การเปรียบเทียบขั้นตอนวิธีการค้นหาแบบไม่ใช้ข้อมูลนำทาง

ตารางที่ 2.9 เปรียบเทียบคุณสมบัติของขั้นตอนวิธีการค้นหาแบบไม่ใช้ข้อมูลนำทาง

ขั้นตอนวิธี	Completeness	Optimality	Time Complexity	Space Complexity
Breadth-First Search	Yes	Yes	$O(b^d)$	$O(b^d)$
Depth-First Search	No	No	$O(b^m)$	$O(bm)$
Uniform-Cost Search	Yes	Yes	Exponential	Exponential
Bidirectional Search	Yes	Yes	$O(b^{d/2})$	$O(b^{d/2})$

จะเห็นได้ว่า ไม่มีขั้นตอนวิธีใดที่ดีที่สุดสำหรับทุกปัญหา โดยแต่ละวิธีมีข้อดีและข้อจำกัดแตกต่างกัน

การค้นหาแบบไม่ใช้ข้อมูลนำทางไม่อาศัยความรู้เพิ่มเติมเกี่ยวกับเป้าหมาย จึงต้องสำรวจสถานะจำนวนมาก ทำให้เกิดการใช้เวลาและหน่วยความจำสูงเมื่อปริภูมิสถานะมีขนาดใหญ่

เพื่อแก้ไขข้อจำกัดดังกล่าว จึงมีการพัฒนาการค้นหาแบบใช้ข้อมูลนำทาง (Informed Search) ซึ่งใช้ฟังก์ชันฮิวริสติก (Heuristic Function) เพื่อช่วยนำทางการค้นหาให้มีประสิทธิภาพมากยิ่งขึ้น ซึ่งจะกล่าวถึงในหัวข้อถัดไป

2.12 การค้นหาแบบใช้ข้อมูลนำทาง

การค้นหาแบบไม่ใช้ข้อมูลนำทาง (Uninformed Search) มีข้อจำกัดสำคัญคือ ขั้นตอนวิธีไม่มีข้อมูลเพิ่มเติมเกี่ยวกับตำแหน่งของเป้าหมาย ทำให้ต้องสำรวจสถานะจำนวนมาก ส่งผลให้ใช้เวลาและหน่วยความจำสูง โดยเฉพาะในปัญหาที่มีปริภูมิสถานะขนาดใหญ่

เพื่อแก้ปัญหาดังกล่าว จึงมีการพัฒนาการค้นหาแบบใช้ข้อมูลนำทาง (Informed Search) หรือการค้นหาแบบฮิวริสติก (Heuristic Search) ซึ่งอาศัยข้อมูลเพิ่มเติมเพื่อช่วยประมาณว่าทิศทางใดมีแนวโน้มที่จะนำไปสู่เป้าหมายได้เร็วที่สุด โดยข้อมูลเพิ่มเติมนี้เรียกว่า ฟังก์ชันฮิวริสติก (Heuristic Function)

การค้นหาแบบใช้ข้อมูลนำทางช่วยลดจำนวนโหนดที่ต้องสำรวจ ทำให้สามารถค้นหาคำตอบได้รวดเร็วกว่าการค้นหาแบบไม่ใช้ข้อมูลนำทาง ขั้นตอนวิธีที่สำคัญในกลุ่มนี้ ได้แก่

- Greedy Best-First Search
- A* Search

2.12.1 ฟังก์ชันฮิวริสติก (Heuristic Function)

ฟังก์ชันฮิวริสติก (Heuristic Function) เป็นฟังก์ชันที่ใช้ประมาณค่าใช้จ่ายจากโหนดปัจจุบันไปยังโหนดเป้าหมาย นิยมเขียนแทนด้วย $h(n)$ เมื่อ

n คือ โหนดปัจจุบัน

$h(n)$ คือ ค่าใช้จ่ายโดยประมาณจากโหนด n ไปยังเป้าหมาย

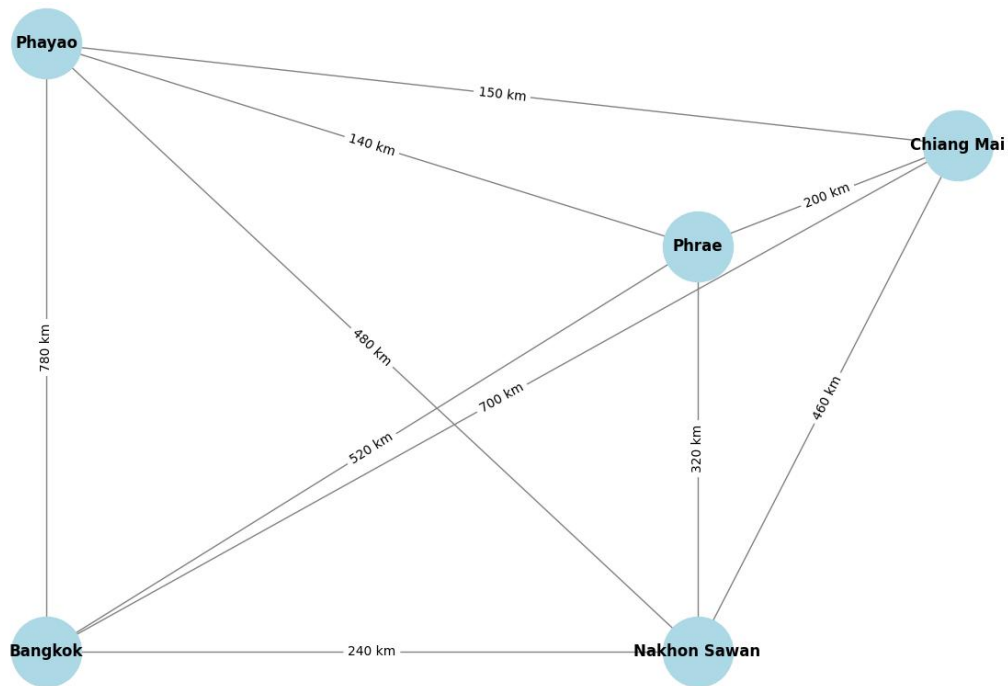
ฟังก์ชันฮิวริสติกไม่ได้เป็นค่าใช้จ่ายจริง แต่เป็นเพียงค่าประมาณที่ช่วยให้ขั้นตอนวิธีสามารถเลือกเส้นทางที่มีแนวโน้มดีที่สุดได้ ตัวอย่างเช่น ในการค้นหาเส้นทางบนแผนที่ดังรูปที่ 2.9 ระหว่างจังหวัดพะเยา (Phayao) เชียงใหม่ (Chiang Mai) แพร่ (Phrae) นครสวรรค์ (Nakhon Sawan) และกรุงเทพฯ (Bangkok) โดยมีเงื่อนไขคือจะต้องผ่านจังหวัดทุกจังหวัดและมีระยะทางสั้นที่สุด

จากตัวอย่างจะพบว่าค่า $h(n)$ สามารถคำนวณจากระยะทางเส้นตรงระหว่างตำแหน่งปัจจุบันที่พะเยากับจุดหมายปลายทางเชียงใหม่ เท่ากับ 150 กิโลเมตร ดังนั้น $h(\text{Chiang Mai}) = 150$ ยิ่งค่า $h(n)$ มีค่าน้อยแสดงว่าโหนดนั้นมีแนวโน้มอยู่ใกล้เป้าหมายมากขึ้น จากตัวอย่างสามารถแสดงค่า $h(n)$ ของแต่ละจังหวัดเชื่อมโยงกับตำแหน่งปัจจุบันดังนี้

$$h(\text{Phrae}) = 140$$

$$h(\text{Nakhon Sawan}) = 480$$

$$h(\text{Bangkok}) = 780$$



รูปที่ 2.9 ตัวอย่างเส้นทางระหว่างจังหวัดแต่ละจังหวัด

2.12.2 ฟังก์ชันการประเมิน (Evaluation Function)

ในการค้นหาแบบใช้ข้อมูลนำทาง จะมีการใช้ฟังก์ชันการประเมิน (Evaluation Function) เขียนแทนด้วย $f(n)$ เพื่อใช้จัดลำดับความสำคัญของโหนดต่าง ๆ โดยทั่วไป $f(n)$ จะมีค่าต่ำสำหรับโหนดที่มีแนวโน้มเป็นคำตอบที่ดี

ขั้นตอนวิธีแต่ละชนิดจะกำหนดรูปแบบของ $f(n)$ แตกต่างกัน ตัวอย่างเช่น

Greedy Best-First Search กำหนด $f(n)=h(n)$

A* Search กำหนด $f(n)=g(n)+h(n)$

เมื่อ $g(n)$ คือ ค่าใช้จ่ายจริงจากจุดเริ่มต้นมายังโหนดปัจจุบัน

$h(n)$ คือ ค่าใช้จ่ายโดยประมาณจากโหนดปัจจุบันไปยังเป้าหมาย

2.12.3 การค้นหาแบบละโมภ (Greedy Best-First Search)

Greedy Best-First Search เป็นขั้นตอนวิธีที่เลือกโหนดซึ่งมีค่า $h(n)$ ต่ำที่สุดก่อน กล่าวคือ $f(n)=h(n)$ หลักการสำคัญของวิธีนี้คือ "เลือกทางเลือกที่สั้นที่สุดในปัจจุบัน" โดยไม่ได้พิจารณาค่าใช้จ่ายที่ผ่านมา

หลักการทำงาน

1. เริ่มต้นจากโหนดเริ่มต้น
2. คำนวณค่า $h(n)$ ของโหนดลูกทั้งหมด

3. เลือกโหนดที่มีค่า $h(n)$ ต่ำที่สุด

4. ทำซ้ำจนกว่าจะพบเป้าหมาย

ตัวอย่างการคำนวณจะใช้ข้อมูลของรูปที่ 2.9

และมีค่า $h(n)$

$h(\text{Chiang Mai}) = 150$

$h(\text{Phrae}) = 140$

$h(\text{Nakhon Sawan}) = 480$

$h(\text{Bangkok}) = 780$

Greedy Best-First Search จะเลือกโหนด Phrae ก่อน เนื่องจากมีค่า $h(n)$ ต่ำที่สุด

ขั้นตอนต่อไปจะเป็นการเปลี่ยนโหนดปัจจุบันเป็น Phrae และตัดโหนดที่เคยผ่านมาแล้วออก นั่นคือ

Phayao ดังนั้นค่า $h(n)$ ของโหนดต่อไปคือ

$h(\text{Chiang Mai}) = 200$

$h(\text{Nakhon Sawan}) = 320$

$h(\text{Bangkok}) = 520$

Greedy Best-First Search จะเลือกโหนด Chiang Mai เนื่องจากมีค่า $h(n)$ ต่ำที่สุด จากนั้นต่อไปจะเลือกโหนด $h(\text{Nakhon Sawan}) = 460$ และ $h(\text{Bangkok}) = 240$

ทำให้ผลรวมของระยะทางจากพะเยาไปยังกรุงเทพฯ โดยผ่าน เชียงใหม่ แพร่ และ นครสวรรค์ คือ $140 + 200 + 460 + 240$ เท่ากับ 1040

ข้อดี

- ค้นหาคำตอบได้รวดเร็ว
- จำนวนโหนดที่ต้องสำรวจมีน้อย

ข้อเสีย

- ไม่รับประกันว่าจะได้คำตอบที่ดีที่สุด จากตัวอย่างข้างต้นคือไม่การันตีว่าเส้นทางทั้งหมดจะเป็นเส้นทางที่ดีที่สุด
- อาจติดอยู่ในคำตอบที่ดีที่สุดเฉพาะที่ (Local Optimum)

ดังนั้น Greedy Search จึงเหมาะสำหรับปัญหาที่ต้องการคำตอบอย่างรวดเร็ว และไม่จำเป็นต้องเป็นคำตอบที่ดีที่สุดเสมอไป

2.12.4 การค้นหาแบบแอสตาร์ (A* Search)

A* Search เป็นขั้นตอนวิธีที่ได้รับความนิยมมากที่สุดในงานด้านปัญญาประดิษฐ์ เนื่องจากรวมข้อดีของ Uniform-Cost Search และ Greedy Best-First Search เข้าด้วยกัน

A* Search ใช้ฟังก์ชันการประเมิน $f(n)=g(n)+h(n)$

โดยที่ $g(n)$ คือ ค่าใช้จ่ายจริงจากจุดเริ่มต้นไปยังโหนดปัจจุบัน

$h(n)$ คือ ค่าใช้จ่ายโดยประมาณจากโหนดปัจจุบันไปยังเป้าหมาย

ดังนั้น A* จะพิจารณาทั้ง

- ระยะทางที่เดินทางมาแล้ว
- ระยะทางที่คาดว่าจะเหลืออยู่

ทำให้สามารถค้นหาคำตอบที่มีประสิทธิภาพสูงได้

หลักการทำงาน

1. เริ่มต้นจากโหนดเริ่มต้น
2. คำนวณค่า $g(n)$ และ $h(n)$
3. คำนวณ $f(n)=g(n)+h(n)$
4. เลือกโหนดที่มีค่า $f(n)$ ต่ำที่สุด
5. ทำซ้ำจนกว่าจะพบเป้าหมาย

ข้อดีของ A* Search

- มีคุณสมบัติการันตีหาคำตอบได้ (Completeness)
- มีคุณสมบัติการได้มาของคำตอบเป็นคำตอบที่ดี (Optimality)
- ลดจำนวนโหนดที่ต้องสำรวจ
- มีประสิทธิภาพสูงกว่าการค้นหาแบบไม่ใช้ข้อมูลนำทาง

ข้อเสีย

- ใช้หน่วยความจำจำนวนมาก
- ประสิทธิภาพขึ้นอยู่กับคุณภาพของฟังก์ชันฮิวริสติก

2.12.5 ฮิวริสติกแบบยอมรับได้ (Admissible Heuristic)

ฟังก์ชันฮิวริสติกที่ดีควรมีคุณสมบัติไม่ประมาณค่ามากเกินไป กล่าวคือ $h(n) \leq h^*(n)$ เมื่อ

$h^*(n)$ คือ ค่าใช้จ่ายจริงต่ำสุดจากโหนดปัจจุบันไปยังเป้าหมาย

ฮิวริสติกที่มีคุณสมบัตินี้เรียกว่า Admissible Heuristic การใช้ Admissible Heuristic ทำให้ A* Search สามารถรับประกันได้ว่าจะค้นพบคำตอบที่ดีที่สุด ตัวอย่างเช่น การคำนวณระยะทางเส้นตรงระหว่างสองเมือง (คำนวณจากฟังก์ชันฮิวริสติก) ต้องมีค่าน้อยกว่าหรือเท่ากับระยะทางจริงบนถนน หากการคำนวณ

จากฟังก์ชันฮิวริสติกมีค่าน้อยกว่าหรือเท่ากับระยะจริง จึงถือว่าเป็น Admissible Heuristic และในทางกลับกันจะถือว่าเป็น Non-admissible Heuristic หากค่าที่คำนวณได้มากกว่าค่าจริง

จากตัวอย่างข้างต้นอาจทำให้สับสน เนื่องจากการสร้างฟังก์ชันฮิวริสติกที่สร้างขึ้นไม่ควรมีตัวอย่างของ Non-admissible Heuristic เกิดขึ้น แต่ในการคำนวณที่ซับซ้อนในกรณีปริภูมิสถานะ (State Space) เยอะมาก การยอมรับให้มี Non-admissible Heuristic แต่สามารถแก้ปัญหาได้ สามารถยอมรับได้มากกว่าการสร้างขั้นตอนวิธีที่เป็น Admissible Heuristic แต่ไม่สามารถหาคำตอบได้ ตัวอย่างเช่น การแก้ปัญหาด้วยการเรียนรู้แบบเครื่องจักรที่นักวิจัยยอมรับให้เกิดข้อผิดพลาดได้บ้างในบางข้อมูล หรือบางจุด

2.12.6 การเปรียบเทียบ Greedy Search และ A* Search

ตารางที่ 2.10 เปรียบเทียบคุณสมบัติของ Greedy Best-First Search และ A* Search

คุณสมบัติ	Greedy Search	A* Search
ฟังก์ชันการประเมิน	$h(n)$	$g(n)+h(n)$
ความเร็วในการค้นหา	สูง	สูง
จำนวนโหนดที่สำรวจ	น้อย	ปานกลาง
Completeness	ไม่เสมอไป	Yes
Optimality	No	Yes
การใช้หน่วยความจำ	ปานกลาง	สูง

จะเห็นได้ว่า Greedy Search มุ่งเน้นการค้นหาคำตอบอย่างรวดเร็ว ในขณะที่ A* Search มุ่งเน้นการค้นหาคำตอบที่ดีที่สุด ดังนั้น A* Search จึงเป็นขั้นตอนวิธีที่ได้รับความนิยมอย่างมากในงานต่าง ๆ เช่น

- ระบบนำทาง GPS
- การวางแผนเส้นทางของหุ่นยนต์
- เกมคอมพิวเตอร์
- การควบคุมยานยนต์อัตโนมัติ

2.13 การประยุกต์ใช้การค้นหา

แนวคิดของการค้นหา (Search) เป็นพื้นฐานสำคัญของปัญญาประดิษฐ์ และถูกนำไปประยุกต์ใช้ในปัญหาต่าง ๆ จำนวนมาก ไม่ว่าจะเป็นการวางแผนเส้นทาง การควบคุมหุ่นยนต์ การจัดตารางเวลา หรือระบบสนับสนุนการตัดสินใจ

แม้ว่าปัญหาแต่ละประเภทจะมีลักษณะแตกต่างกัน แต่ล้วนสามารถแปลงให้อยู่ในรูปของปริภูมิสถานะ (State Space) และใช้ขั้นตอนวิธีการค้นหาเพื่อค้นหาคำตอบที่เหมาะสมที่สุด

ตัวอย่างของปัญหาที่ได้รับความนิยมในการศึกษาปัญญาประดิษฐ์ ได้แก่

- ปัญหาการทอนเงิน
- ปัญหาการเดินทางของเซลแมน
- การค้นหาเส้นทางบนแผนที่

2.13.1 ปัญหาการทอนเงิน (Coin Change Problem)

ปัญหาการทอนเงินเป็นปัญหาคณิตศาสตร์ที่มีวัตถุประสงค์เพื่อค้นหาชุดของเหรียญหรือธนบัตรที่สามารถใช้ทอนเงินได้ โดยใช้จำนวนเหรียญหรือธนบัตรให้น้อยที่สุด

สมมติว่าต้องการทอนเงินจำนวน 378 บาท

และมีธนบัตรและเหรียญดังนี้

- 100 บาท
- 50 บาท
- 20 บาท
- 10 บาท
- 5 บาท
- 2 บาท
- 1 บาท

ขั้นตอนวิธีแบบละโมภ (Greedy Algorithm) จะเลือกธนบัตรหรือเหรียญที่มีค่ามากที่สุดก่อน ดังนั้น

$$378 \text{ บาท} = 100 + 100 + 100 + 50 + 20 + 5 + 2 + 1$$

รวมทั้งหมด 8 หน่วย

สำหรับระบบสกุลเงินไทยในปัจจุบัน วิธีแบบละโมภสามารถให้คำตอบที่ดีที่สุดได้

อย่างไรก็ตาม ในบางกรณี ขั้นตอนวิธีแบบละโมภอาจไม่สามารถรับประกันคำตอบที่ดีที่สุดได้ ตัวอย่างเช่น มีเหรียญ

- 10 บาท
- 7 บาท
- 1 บาท

หากต้องการทอนเงิน 14 บาท

Greedy Algorithm จะเลือก

$$10 + 1 + 1 + 1 + 1$$

รวม 5 เหรียญ

แต่คำตอบที่ดีที่สุดคือ

7 + 7

ซึ่งใช้เพียง 2 เหยี่ยว

ดังนั้น ขั้นตอนวิธีแบบละโมภจึงไม่รับประกันว่าจะให้คำตอบที่ดีที่สุดเสมอไป

2.13.2 ปัญหาการเดินทางของเซลแมน (Travelling Salesman Problem)

ปัญหาการเดินทางของเซลแมน หรือ Travelling Salesman Problem (TSP) เป็นปัญหาคลาสสิกในงานด้านการหาค่าเหมาะสม (Optimization) วัตถุประสงค์ของปัญหาคือ ค้นหาเส้นทางที่มีระยะทางรวมสั้นที่สุด โดยต้องเดินทางผ่านทุกเมืองเพียงหนึ่งครั้ง และกลับมายังจุดเริ่มต้น

ตัวอย่างเช่น กำหนดให้มีเมือง

- กรุงเทพมหานคร
- นครสวรรค์
- แพร่
- พะเยา
- เชียงใหม่

หากเริ่มต้นที่กรุงเทพมหานคร เส้นทางที่เป็นไปได้มีจำนวนหลายรูปแบบ เช่น

กรุงเทพฯ → นครสวรรค์ → แพร่ → พะเยา → เชียงใหม่ → กรุงเทพฯ

หรือ

กรุงเทพฯ → เชียงใหม่ → พะเยา → แพร่ → นครสวรรค์ → กรุงเทพฯ

เมื่อจำนวนเมืองเพิ่มขึ้น จำนวนเส้นทางที่เป็นไปได้จะเพิ่มขึ้นอย่างรวดเร็ว

สำหรับเมืองจำนวน n เมือง จำนวนเส้นทางที่เป็นไปได้เท่ากับ $(n - 1)!$

ดังนั้น เมื่อจำนวนเมืองมีค่ามาก การค้นหาทุกความเป็นไปได้จะใช้เวลาอย่างมาก

ปัญหาการเดินทางของเซลแมนจัดอยู่ในกลุ่ม NP-Complete และเป็นหนึ่งในปัญหาที่ได้รับการศึกษาอย่างกว้างขวางในงานด้านปัญญาประดิษฐ์และการหาค่าเหมาะสม

2.13.3 การค้นหาเส้นทางบนแผนที่ (Route Finding Problem)

การค้นหาเส้นทางบนแผนที่เป็นตัวอย่างที่สำคัญของการประยุกต์ใช้การค้นหาในชีวิตประจำวัน ตัวอย่างเช่น ระบบนำทาง GPS Google Maps ระบบนำทางของรถยนต์อัตโนมัติ ปัญหาประเภทนี้สามารถแทนด้วยกราฟ (Graph) โดย

- เมืองหรือจุดต่าง ๆ แทนด้วยโหนด (Node)
- ถนนแทนด้วยเส้นเชื่อม (Edge)
- ระยะทางหรือเวลาเดินทางแทนด้วยค่าใช้จ่าย (Cost)

เป้าหมายคือการค้นหาเส้นทางที่มีค่าใช้จ่ายต่ำที่สุดจากจุดเริ่มต้นไปยังจุดหมายปลายทาง ขั้นตอนวิธีที่นิยมใช้ ได้แก่

- Uniform-Cost Search
- Dijkstra Algorithm
- Greedy Best-First Search
- A* Search

โดยเฉพาะ A* Search ซึ่งได้รับความนิยมอย่างมาก เนื่องจากสามารถค้นหาคำตอบได้รวดเร็วและมีประสิทธิภาพสูง

2.13.4 การประยุกต์ใช้ในหุ่นยนต์

หุ่นยนต์อัตโนมัติจำเป็นต้องวางแผนเส้นทางเพื่อหลีกเลี่ยงสิ่งกีดขวางและเดินทางไปยังตำแหน่งเป้าหมาย ตัวอย่างเช่น

- หุ่นยนต์ดูดฝุ่นอัตโนมัติ
- หุ่นยนต์ส่งสินค้า
- แขนกลในโรงงานอุตสาหกรรม
- รถยนต์ขับเคลื่อนอัตโนมัติ

ปัญหาเหล่านี้มักใช้

- Breadth-First Search
- A* Search
- Dijkstra Algorithm

เพื่อค้นหาเส้นทางที่เหมาะสมที่สุด

2.13.5 การประยุกต์ใช้ในเกมคอมพิวเตอร์

เกมคอมพิวเตอร์จำนวนมากใช้ขั้นตอนวิธีการค้นหาเพื่อควบคุมพฤติกรรมของตัวละครที่ควบคุมโดยคอมพิวเตอร์ (Non-Player Character: NPC) ตัวอย่างเช่น

- เกมวางแผน (Strategy Game)
- เกมต่อสู้
- เกมผจญภัย
- เกมกระดาน

การค้นหาถูกนำมาใช้ในงานต่าง ๆ เช่น

- การหาเส้นทางของตัวละคร
- การวางแผนการโจมตี

- การหลบหลีกศัตรู
ขั้นตอนวิธีที่นิยมใช้ ได้แก่
- A* Search
- Minimax Algorithm
- Alpha-Beta Pruning

2.13.6 การประยุกต์ใช้ในปัญญาประดิษฐ์สมัยใหม่

แม้ว่าปัญญาประดิษฐ์ในปัจจุบันจะพัฒนาไปสู่การเรียนรู้ของเครื่องและการเรียนรู้เชิงลึก แต่แนวคิดของการค้นหา ยังคงมีบทบาทสำคัญ ตัวอย่างเช่น

- ระบบแนะนำสินค้า
- ระบบตอบคำถามอัตโนมัติ
- Large Language Model
- การวางแผนของหุ่นยนต์
- Reinforcement Learning

จะเห็นว่าแนวคิดของการค้นหาไม่ได้จำกัดอยู่เฉพาะปัญหาคลาสสิกเท่านั้น แต่ยังเป็นพื้นฐานสำคัญของระบบปัญญาประดิษฐ์สมัยใหม่อีกด้วย ดังนั้น การศึกษาขั้นตอนวิธีการค้นหาจึงถือเป็นพื้นฐานสำคัญของการศึกษาปัญญาประดิษฐ์ และเป็นรากฐานสำหรับการเรียนรู้หัวข้อขั้นสูง เช่น การค้นหาแบบมีคู่แข่ง (Adversarial Search) การหาค่าเหมาะสม (Optimization) และการเรียนรู้ของเครื่อง (Machine Learning)

แบบฝึกหัดท้ายบทที่ 2

1. จงอธิบายความหมายของตัวแทน (Agent) และตัวแทนที่มีเหตุผล (Rational Agent) พร้อมยกตัวอย่างประกอบ
2. ตัวรับข้อมูล (Sensors) และตัวกระทำ (Actuators) มีหน้าที่แตกต่างกันอย่างไร จงอธิบายพร้อมยกตัวอย่าง
3. เพราะเหตุใดการกำหนดเกณฑ์การวัดประสิทธิภาพ (Performance Measure) จึงมีความสำคัญต่อการออกแบบระบบปัญญาประดิษฐ์
4. จงอธิบายความแตกต่างระหว่าง Rationality และ Omniscience
5. การแทนความรู้แบบ Atomic Representation, Factored Representation และ Structured Representation แตกต่างกันอย่างไร
6. จงอธิบายขั้นตอนการทำงานของตัวแทนแก้ปัญหา (Problem-Solving Agent)
7. องค์ประกอบของปัญหาที่นิยามไว้อย่างชัดเจน (Well-defined Problem) ประกอบด้วยอะไรบ้าง พร้อมอธิบายหน้าที่ของแต่ละองค์ประกอบ
8. จงอธิบายความหมายของ Initial State, Goal State และ Path Cost พร้อมยกตัวอย่างประกอบ
9. ปัญหา 8 ตัวเลข (8-Puzzle Problem) มีลักษณะอย่างไร และสามารถแทนปัญหาให้อยู่ในรูปแบบมาตรฐานได้อย่างไร
10. จงอธิบายเงื่อนไขของปัญหา 8 ราชีนี (8-Queens Problem)
11. ปัญหาแบบ Deterministic Problem แตกต่างจาก Non-Deterministic Problem อย่างไร พร้อมยกตัวอย่าง
12. ปัญหาแบบ Stochastic Problem มีลักษณะอย่างไร และแตกต่างจาก Non-Deterministic Problem อย่างไร
13. จงอธิบายความหมายของ Completeness และ Optimality ของขั้นตอนวิธีการค้นหา
14. Time Complexity และ Space Complexity คืออะไร และมีความสำคัญอย่างไรต่อการวิเคราะห์ขั้นตอนวิธี
15. จงอธิบายความหมายของสัญกรณ์โอใหญ่ (Big-O Notation)
16. กลุ่มปัญหา P, NP, NP-Complete และ NP-Hard แตกต่างกันอย่างไร
17. เพราะเหตุใดปัญหาการเดินทางของเซลแมน (Travelling Salesman Problem) จึงจัดเป็นปัญหาที่มีความซับซ้อนสูง
18. จงอธิบายหลักการทำงานของ Breadth-First Search
19. Breadth-First Search มีข้อดีและข้อจำกัดอย่างไร

- 20.จงอธิบายหลักการทำงานของ Depth-First Search และเปรียบเทียบกับ Breadth-First Search
- 21.Uniform-Cost Search แตกต่างจาก Breadth-First Search อย่างไร
- 22.เพราะเหตุใด Bidirectional Search จึงสามารถลดเวลาในการค้นหาได้
- 23.ฟังก์ชันฮิวริสติก (Heuristic Function) คืออะไร และมีบทบาทอย่างไรในการค้นหาแบบใช้ข้อมูลนำทาง
- 24.Greedy Best-First Search และ A* Search แตกต่างกันอย่างไรร
- 25.เพราะเหตุใด A* Search จึงได้รับความนิยมในการประยุกต์ใช้งานจริง
- 26.Admissible Heuristic คืออะไร และมีความสำคัญต่อ A* Search อย่างไร
- 27.จงยกตัวอย่างการประยุกต์ใช้ขั้นตอนวิธีการค้นหาในชีวิตประจำวันอย่างน้อย 3 ตัวอย่าง
- 28.การค้นหาเส้นทางบนแผนที่สามารถแทนปัญหาให้อยู่ในรูปของกราฟได้อย่างไร
- 29.การค้นหาในปัญญาประดิษฐ์มีบทบาทสำคัญต่อระบบปัญญาประดิษฐ์สมัยใหม่อย่างไร
- 30.จากเนื้อหาในบทนี้ นักศึกษาคิดว่าขั้นตอนวิธีการค้นหาแบบใดเหมาะสมที่สุดสำหรับการพัฒนาระบบนำทางรถยนต์อัตโนมัติ พร้อมอธิบายเหตุผล

อ้างอิง

1. Cook, S. A. (1971). The complexity of theorem-proving procedures. Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC), 151–158.
<https://doi.org/10.1145/800157.805047>
2. Karp, R. M. (1972). Reducibility among combinatorial problems. In R. E. Miller & J. W. Thatcher (Eds.), Complexity of Computer Computations (pp. 85–103). Springer.
https://doi.org/10.1007/978-1-4684-2001-2_9
3. Sipser, M. (2012). Introduction to the theory of computation (3rd ed.). Cengage Learning.
(สำหรับภาพรวมทฤษฎี P, NP, NP-Complete และ NP-Hard)